I N D E X

NAME: Shelke prakik Anandrao. ROLL NO.:

STD.: DIV./SEC.: SUBJECT: "AI Engineer."

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
		।। श्री गणेशाय नमः ।।		

Date .....
Page .....

* session 1:- Python programming and programming language introduction:

1) Why do We need programming language?
We need programming language to communicate with computers and give them instructions. they help us create software, websites, mobile apps, games and automate tasks.
e.x:- programming language allows us to give instruction to a computer and build different type of application.

2) Why is python so popular?
"python is popular because of it's simple syntax, large number of libraries, strong community support, and wide range of application such Web development, data science, AI and automation.

3. What is the history of Python version?
Python was created by Guido Van Rossum and released in 1991. python 2 was released in 2000, and python 3 was released in 2008. Today, Python 3 is the standard version used for development.

4. What does "open-source" means?
open source means the source code is publicly available and can be used, modified and shared accoeding to it's license.

5. Which companies use python, and What do they use it for?
" Companies such as Google, Netflix, Instagram, Spotify and Dropbox use python for areas like backend

development, automation, data analysis & ML.

6. Where is python commonly used in the IT industry
1. Web development: Django, flask
2. Data Analysis: pandas, NumPy
3. Machine Learning: scikit-learn
4. AI.
5. Data Visualization - Matplotlib.
6. Cyber security.
7. Web scraping.

7. What are the key features of python?
"The key features of python are simple syntax, readability, portability, open-source nature, object oriented programming, dynamic typing & a large collection of libraries".

* session 2:-

1. What is python?
python is high-level, interpreted, general purpose programming language. It has simple and readable syntax and is widely used for Web development, data analysis, AI/ML, automation & testing.
e.g. print("Hello, world").
Note:- python is called a high-level language because it's syntax is easy for humans to understand and hides low-level computer details. It is called general purpose because it can be used for many tasks such as web development, AI, data analysis & testing.

Bytecode :- Bytecode is low-level, platform independent set of instructions created by the python interpreter to executed by python virtual machine.

Date .....
Page .....

2. Who created python?
python was created by Guido van Rossum.
- He started developing python in the late 1980s & released it publicly in 1991.

3. Difference betn python 2 & python 3

python 2	python 3
- oldee version	- Modeen version.
- Released in 2000	- Released in 2008.
- officially discontinued in 2020	- Actively used & developed.
- print "Hello"	- print("Hello")

4. What is Bytecode?
- bytecode is an intermediate form of python code generated from the source code. It is not directly understood by the computers CPU.
- python commonly stores compiled bytecode in .pyc file inside the __pycache__ folder.
- flow

python source code (.py)
↓
bytecode
↓
PVM.

5. What is PVM (python virtual machine)?
→ It is the part of the python runtime that executes python bytecode.

- Simple flow:

Source code
↓
Byte code
↓
PVM
↓
output

- easy :- PVM is like a machine that understand & executes python bytecode.

8. explain the internal Working of python?
1. We Write python source code in a .py file.
2. Python compiles the code into bytecode.
3. The python runtime executes the bytecode using the PVM.
4. The program produces the required output.

9. Source code vs bytecode vs Machine code.

Source code	bytecode	Machine code.
① Written by programmer	① Intermediate code	① CPU instructions
② human-readable.	② Generally not human friendly	② very low level
③ e.x :- .py	③ e.x :- .pyc	③ binary instructions.

8. How do you check the installed python version.
- python --version
  python3 -- version.

9. What is extension of a python file?.
- .py

* Session 3 :- Identifier, Keyword, Variable, Data type.

1. What is an identifier in python?
→ An identifier is the name used to identify a variable, function, class, object etc.
ex: name='pratik'
age=24
Here name, & age are identifiers.

2. What are the rules for creating identifiers in python.
→ 1. It can contain letter (A-Z, a-z), digit (0-9) & underscore -.
2. It cannot start with a digit.
3. It cannot contain space or special character like @, $, %
4. It cannot be python keyword.
5. python identifier are case sensitive.

3. Can we use keywords as variable names? why?
→ No. we cannot use keywords as variable names because keywords have a special meaning in python syntax.
ex :- for=10
class=20 } # Invalid.

4. What is meant by python being case-sensitive?

→ Python treats uppercase & lowercase letters as different.

Ex
```
name = "Pratik"
Name = "Rahul"
print(name)
print(Name)
```
Here name & Name are two different variable.

5. What are python keywords?

- Keywords are reserved words that have a special meaning in python.
- ex :- if, else, for, while, class, def, True, None.
- We can not normally use them as identifiers.

6. Why can't we use for, if, or class as variable name?

→ because they are python keywords used for specific programming purposes.

- example.

```
if age > 18:
    print("Adult")
```

7. What is the difference bet^n True & "True"?

→

True	"True"
- Boolean value	- string
- Represents truth	- Represents text
- Type is bool	- Type is str

8. What are basic data type in python?

→
1. int
2. float
3. complex
4. bool
5. string
6. byte

} Basic data type.

2. containee Data type:
list, tuple, set, dictionary..

3. User-defined data type:
class

* basic Data type

int	float	complex
1> decimal (base 10)	1> Decimal form	format=
2. binary (base 2)	ex = Pi = 3.14	real + imaginary j
3> octal (base 8)	Price = 99.99	ex. c1 = 3+2j
4. hexadecimal (base16)	2>exponential form	c2 = 1+4j
	x = 3.14 e2	
	$3.14 * 10^2$	
	3.14.0.	

Boolean type
① True
② false.

string
① quote = ' '
= " "
= """ """

9. What is the difference bet^n int & float
→ • int :- stores whole number
• float :- stores number with decimal points.
ex :- a=10 # int
b=10.5 # float.

10. What is a complex number in python?
→ A complex number contains a real part & an imaginary part.
-python uses j for the imaginar part.

- ex:- X = 3+4j
  3 → real part
  4j → imaginary part.

11. What are container data type in python? Name them
→ container data types are used to store multiple value.
- common container type are:
  - List - []
  - Tuple - ()
  - set - {}
  - Dictionary - {key : value}

12. What is boolean type?
→ Boolean (bool) is a data type used to represent True or false value.
- example:
  is_student = True.

13. What are the possible value of Boolean variable?
→ There are only two value:
  True
  false.

14. What is a string in python?
→ A string is a sequence of characters enclosed in single, double or triple quotes.
- ex:- name = "pratik"
  city = 'pune'.

15. What is dynamic typing in python?
Dynamic typing means we do not need to specify the variable's data type explicitly.

Date .....
Page .....

python determines the type automatically at runtime.

e.x :-

```
x=10
```

Here python automatically understands that x is the an integer.

```
x = "Prathk"
```

Now x is refers to string.

16. can the same variable hold different type of values in python? explain with an example.

→ yes, Because python is dynamically typed, a variable can refer to value of different types at different time.

ex

```
x=10
print(type(x))  # int
x = "python"
print(type(x))  # str
x=10.5
print(type(x))  # float
```

17. What is the difference bern decimal, binary, octal & hexadecimal number?

Number system	Base	Digits
Decimal	10	0-9
binary	2	0-1
octal	8	0-7
Hexdecimal	16	0-9, A-F.

ex: the decimal number is.

Decimal = 10 Hexadecimal → A.

binary = 1010

octal = 12

Date
Page

18. How do we represent binary, octal and hexadecimal numbers in python?

→ python provides prefixes for these number system.

binary = 0b1010	# 10
octal = 0o12	# 10
hexadecimal = 0xA	# 10

19) What are the prefixes used binary, octal, & hexadecimal number in python?

Number system	prefix	example
binary	0b or 0B	0b1010
octal	0o or 0O	0o12
Hexadecimal	0x or 0X	0xA.

20) What is Data type?

→ A data type is python defines the type of value stored in a variable and determines what operation can be performed on that value.

-ex:- x=10
print(type(x))
o/p <class 'int'>

* session 4: Variable assignment, operator (Arithmatical operator).

1. What is variable in python?

→ A variable is a name used to store or refer to a value in python.

ex. name = "pratik"
age = 24

here name and age are variable.

2. Can multiple variable be assigned values in a single python statement?

→ Yes, python support multiple assignment.

a, b, c = 10, 20, 30

here a = 10
b = 20
c = 30

We can also assign the same value to multiple variables:

a = b = c = 10

3. What is the difference between:

a = 10 and a = b = 10
b = 10

→ - both assign the value 10 to a and b
- Difference is mainly is assigment style.
- In python variable refer to object; a = b = 10 markes both names refer to same integer object.

4. What happens if the number of variables and value do not match?

→ python gives a valueError because the number of variables must match the number of values.

*ex:- a, b = 10, 20, 30

o/p: valueError: too many values to unpack.

* Another example:-

a, b, c = 10, 20.

o/p: valueError: too many values to unpack.

5. Can variables of different data types be assigned in the same statement?

→ Yes,

name, age, marks = "pratik", 24, 85.5

Here.

name = pratik 'str'

age = int

marks = float

Arithmetic operator

6. What is the difference betn / and // in python?

→

/	//
1) Division operator	1) Floor division operator.
2) Return division result.	2) Return quotient rounded down.
3) usually returns float	3) can return int when both operands are integer

ex: print (10/3) # 3.33333333

print (10//3) # 3

7. What is the purpose of the % operator?

→ - The % operator is called the modulus operator.

- It returns the remainder after division.

ex:- print (10 % 3)

1

$$3 \overline{)10}\ 3 \quad -9 \quad 1 \leftarrow \%p$$

8. What type of result does the / operator normally produce?

→ The / operator normally produces a float result, even when both operands are integers

ex:- result = 10/2
print(result) # 5.0
print(type(result)) # <class 'float'>

9. What is the difference bet^n multiplication & floor division:-

→ *multiplication:- * calculate the product.
print(5*3)
# 15
* floor division:- // divides two numbers and returns the result rounded down.
print(10//3)
# 3.

10. Where can the modulus operator be useful in real-world programming:

→ The % operator is useful when we need to find a remainder.
ex:-
1) checking even/odd numbers:
num=10
if num % 2 == 0:
print("even")
2) checking divisibility:
if num % 5 == 0:
print("Divisible by 5").

11) What happens when you performs floor divisio on two integers?

→ It returns the floor of division result.
print(10//3)
#3.

Important :- Floor division means rounding down, not simply removing decimal.

ex :- print (-10 // 3)

o/p = -4

12. can arithmetic operators be used with data type other than integers.

→ Yes. Arithmetic operators can work with several data types.

* float :-
print (5.5 + 2.5)

* complex number :
print ((2+3j) + (1+2j))

* String with +
print ("hello" + "python")
⇒ hello python.

However, not every arithmetic operator works with every data type.

13. What is the purpose of id() in Python?

→ id() return an integer that identifies an object during it's lifetime.

ex :- x=10
print(id(x))

It gives a number as :
140735.....

- id() is built-in python function that return the identity of an object.

14. If two variable contain the same value, will their id() always be the same.

→ No, not always
example:
```
a=10
b=10
print (a==b)   # True.
print (a is b)   # often True for small integers.
print (id(a))
print (id(b))
```
* Remember
- == → compares values
- is → compares object identity.

15. What is operator precedence in python?
→ operator precedence determines the order in which operators are evaluated in an expression.
- for example:-
```
result = 10+5*2
# 20
```

16. Why is operator precedence important when writing arithmetic expressions?
→ It tells python which operation should be performs first.

17. Which has higher precedence : multiplication or addition?
→ multiplication (*) has higher predence than addition (+).

18. What happens when two operators have the same precedence?
→

Python uses associativity to decide the order of evaluation.
-for most acithmetic operators, evaluation is left to right.

ex.

```
print(20-5+2)
```

both - and + have the same precedence, so python evaluates from left to right.

20 - 5 = 15

15 + 2 = 17

Result = 17.

* Session 5: String (indexing, slicing)

1. What is a string in python?

→ A string is a sequence of Unicode character enclo in single, double, triple quotes.

- ex name: "pratik".

2. What are unicode characters?

→ Unicode is standard that represents character from almost all languages, Symbols & emojis;

3. Why does python use Unicode?

→ Python uses Unicode so that strings can support characters from many different language & Symbols.

ex

```
name="@123ABC"
```

- Unicode allows python string to represent text from different language & character sets.

Date .....
Page .....

4. How can you create a string in python?

→ We can create string using:

- Single quotes: ' ' a = 'Python'
- Double quotes: " " b = "Hello"
- Triple single quotes: ''' ''' c = '''This is a string'''
- Triple double quotes: """ """

ex d = """This also a string""".

5. What is the different betn single quotes & double quotes?

→ There is no major difference. Both create string.

a = 'Hello'

b = "Hello"

Both are string.

- Double quote can be useful when the string contains a single quote:

text = "I'm a student"

- Single quote can be useful when the string contain a double quote:

text = 'He said "Hello"'.

6. How do you access character in a string?

→ We access character using indexing.

ex :- text = "python"

print(text[0]) # p

print(text[2]) # t.

7. What is indexing?

→ Indexing means accessing an individual character from a string using it's position.

text = "python"

print(text[1]) # y.

8. From Where does positive indexing start?

→ positive indexing starts from 0, from the left side
- for "python"

p	y	t	h	o	n
0	1	2	3	4	5

So:

```
text[0]   # p
```

9. From Where does negative indexing start?

→ Negative indexing start from -1; from the right side.

p	y	t	h	o	n
-6	-5	-4	-3	-2	-1

So

```
text[-1]  # n
```

10. Why is negative indexing useful?

→ Negative indexing allows us to access characters from the end of the string. Without calculating the last position.

ex
```
text = "python"
print(text[-1]) # n
print(text[-2]) # o.
```

11. What happens When you try to access an invalid string index?

→ python raises an IndexError.

ex
```
text = "python"
print(text[10])
# IndexError : string index out of range.
```

12. .can indexing be used to modify a string? why?

→ NO

string are immutable, meaning their characters cannot be changed directly.

- ex: text = "python"

```
text[0] = "J"  # error.
```

This given: TypeError.

- Instead, we create a new string.

```
text = "python"
text = "J" + text[1:]
print(text)  # Jpython.
```

13. Can a string contain numbers, spaces & special characters?

→ Yes,

```
text = "Python 123 @ 2026 !"
```

A string can contain:

letter, Number, spaces, special character, symbol, unicode characters.

14. Does python have a separate character data type?

→ NO,

Python does not have a separate char data type.

- A single character is simply a string of length 1.

-example:-

```
x = "A"
print(type(x))  # <class 'str'>
```

15. What is the difference betn positive and negative indexing?

Positive Indexing	Negative Indexing.
1) start from 0	1) starts from -1
2) Moves left to right	2) Moves right to left.
3) text[0] → first character.	3) text[-1] → last charac

ex
```
text="Python"
print(text[0])  # P.
print(text[-1])  # n.
```

16. Why are string widely used in python?

→ string are widely used because they are needed to work with text & characters.

⇒ common uses:

- Names, message, user i/p, file data, URL's, Emails passwords, Data analysis.

string slicing

17. What is string slicing?

→ slicing means extracting a part of a string.

- example:

```
text="python"
print(text[1:4])
# yth.
```

18. What is the syntax of string slicing?

→ The syntax is:

```
string[start:stop:step]
```

example:

```
text[1:5:2]
```

19. What do start, stop and step mean?

→ text[start. stop: step]

- start → Where slicing begins.
- stop → Where slicing ends.
- step → how many positions to move at a time.

example: text = "python"
print (text [0:6:2])

pto

position 0 - 2 - 4

20. Is the stop index included in slicing:-

→ No;

The stop index is excluded:

- text = "python"
print (text [1:4])

indexes 1, 2, 3 are included, but index 4 is not.

olp :- yth

21. What is the default value of the start index?

→ The default value of start index is 0, The default starts is the beginning of the string.

e.x: text = "python"
print (text [:3])

This equivalent to
print [0:3]

pyt.

22. What is the default value of the stop index?

→ The default stop is the end of the string.

e.x

```
text = "python"
print(text[2:])  # thon
```

23. What is the default value of the step?
→ The default step is 1.
e.x :- text[1:5]
is equivalent to
text[1:5:1]

24. What happens when the start index is omitted?
→ Python starts slicing from the beginning.
e.g :- text = "python"
print(text[:4])
o/p :- pyth

25. What happens when the stop index omitted?
→ Python continues slicing until the end.
```
text = "python"
print(text[2:])
 # thon.
```

26. What happen when the step value is negative?
→ A negative step moves through the string from right to left.
- example:
```
text = "python"
print(text[::-1])
# nohtyp.
```

27. How can you reverse a string using slicing?
→ string[::-1]

Date .....
Page .....

28. can slicing modify the original string?

→ No

strings are immutable.

```
text = "python"
part = text[0:3]
print(text)
# python.
```

The original string remains unchanged.

29. Does string slicing create a new string.

→ Yes,

slicing a string produces a new string containing the selected character.

ex:-
```
text = "python"
part = text[0:3]
print(part)
# pyt
```

30. What is the difference betn. indexing & slicing?

indexing	slicing
1) Accesses one character	1) extracts multiple character.
2) Uses one index	2) uses range
3) text[2]	3) text[2:5]
4) Return a string of length 1	4) Returns a substring.

ex:
```
text = "python"
print(text[2])   # t
print(text[2:5])  # tho
```

31. What happens if the start index is greater than the stop index?

→ With the default positive step, it normally return
an empty string.
ex: text = "python"
print(text[5:2])
# ''
But with a negative step, it can work in reverse
print(text[5:2:-1]) # noh

32. What happens when the step value is 0 in slicing?
→ A step of 0 is not allowed.
ex:- text = "python"
print(text[1:5:0])
# ValueError: slice step cannot be zero.
- step can be positive or negative ~~string~~ but not

33. Can negative indexing be used in string slicing?
→ Yes,
ex:- text = "python"
print(text[-4:-1])
# tho
- Negative indexes are useful when working
from the end of the string.

34. explain the difference betn text[2] & text[2:3]
→ Suppose
text = "python"
① text[2]
Return one character
print(text[2])
# t
② text[2:3]
- Return substring
t.

b] search method :- find(), replace(), return index.
c] Trim whitespace method :- lstrip(), rstrip(), strip()
d] split Method :- split(), partition() {in review}
e] case sensitive Methode :- lower(), upper(), conversion capitalize(), title(), swapcase()

Date .....
Page .....

35. explain the difference beth text[:], text[::] and text[::-1]

1] text[:] → return the whole string
2] text[::] → Aslo return the whole string, because start + stop are default + step is 1.
3) text[::-1] : Return the string in reverse order because the step is -1

* session 6: string properties, function, methode:
1> properties :- immutability, concatention, repeation, membership property (in or not in).
2> function :- len(), min(), max()
3> methode :- ~~isalpha()~~.
↳ @ content test methods :- isalpha(), isdigit(), isalnum() { isalphanumerical & check all character is alphabets -o digits}, islower(), isupper(), startswith(), endswith(),

1. What is meant by string immutability in python?
→ string immutability means that once a string is created it's characters cannot be changed.
ex :-
text = "python"
We cannot directly change "p" to "J"
(string can be override).

2. can We change an individual character of a string after creating it? why?
→ No, We cannot change an individual character becauses strings aee immutable.
- ex text = "python"
text[0] = "J" # error

3. What happens if we try to modify an individual character of a string?

→ Python raises a TypeError.

ex: text = "python"
text[0] = "J"

o/p: TypeError: 'str' object does not support item assignment.

4. If string are immutable, how can we assign a new string to the same variable?

→ We can assign a completely new string to the variable.

ex text = "python"
text = "Java"
print(text) # Java.

—— String operators ——

5. What is string concatenation, & which operator is used for it?

→ string concatenation means joining two or more strings together.

- The + operator is used.
- example:

first = "Hello"
second = "python"
result = first + " " + second
print (result)
Hello python.

6. What is string repetition, & which operator is used for it?

→ String repetition means repeating a string multiple times
- The * operator is used.
- example :- text = "Hi"
print (text * 3)

Hi Hi Hi

7. What is the difference bet^n the + operator and the * operator When Working with string?

→

+	*
1) used for concatenation	1) used for repetition
2) joins strings	2) Repeats a strings
3) "Hello" + "world"	3) "Hi" * 3
4) Helloworld	4) HiHiHi

Membership testing

8. What is membership testing in python string?
→ Membership testing checks whether a character or substring exists inside a string.
- python uses:
  - in
  - not in

ex :- text = "python"
print ("p" in text) # true
print ("java" in text) # false

9. What is the difference bet^n the in and not in operators?
→ 1) in
checks whether something exist in the string.
ex : print ("py" in "python") # True.
2) not in :-
check whether something does not exist in the string
ex :- print ("java" not in "python")
True

Date
Page

10. What type of result do the in and not in operators return?

→ Both return a Boolean value.
True or False.

ex:
```
text="python"
print("p" in text) # True
print("x" not in text) # True.
```

11. Is string membership testing case-sensitive?

→ Yes, string membership testing in case-sensitive
- example:
```
text="python"
print('p' in text) # true
print("p" in text) # False
```
"p" and "p" are treated as different characters.

|| len(), min(), max() ||

12. What is the purpose of the len() function when working with strings?

→ len() return the number of character in a string
ex:
```
text="python"
print(len(text))
# 6
```

13. Does len() count spaces present in a string?

→ Yes, spaces are also counted as characters?
```
text="Hello world"
print(len(text))
# 11
```

14. What is purpose of the min() and max() functions When used with strings?

→ min() returns the smallest character, while max() returns the largest character according to character ordering.

ex: text = "abcde"
print(min(text)) # a
print(max(text)) # e

15. on What basis does python determine the minimum & maximum character of a string?

→ python determines them based on the unicode code point values of the characters.
for simple english letters:
'a' < 'b' < 'c' < ... < 'z'
uppercase and lowercase letter have different unicode values.

ex:- print(min("abc")) # a.
print(max("abc")) # c.

String Methods

16. What is the difference bet^n a built-in function & a string method?

→

built-in function	string method.
1) General python function	1) specifically works with strings.
2) called directly	2) called using a string object
3) e.x :- len(text)	3) e.x :- text.upper()

* built in function syntax:
fun-name (string-variable-name)
e.x :- len (name)
* string methods:
syntax :- string-variable-name . method-name()
e.x :- name . upper().

17. What is a string method, and what is the general syntax for calling a string method?
→ - A string method is a built-in operation provided by python to perform tasks on steings.
- General syntax:
string.method()
- e.x :- text = "python"
print(text.upper()) # PYTHON
* some common string methods:
upper(), lower(), isalpha(), is digit(), isalnum(), steip(), replace() split().

——— || character checking methods || ———

18. What is the purpose of the isalpha() methode?
→ isalpha() checks whether all characters in a string, are alphabetic letters.
- It returns:
• True → if all characters are letters and the string is not empty.
• false → if the string contains numbers, spaces or special characters.
- e.x:
print ("python" .isalpha()) # True.
print ("python 123". isalpha()) # false.

19. What is the purpose of the isdigit() method?

→ isdigit() checks whether all characters in a string are digits.

- It returns:
- 'True': if all characters are digit and the string is not empty.
- false → if letters, spaces or other character are present

ex: print("12345".isdigit()) # True.
print("1234abc".isdigit()) # False.

20. What is the difference betn isalpha(), isdigit() & isalnum()?

Method	checks for	example.
isalpha()	letters only	"Python"
isdigit()	Digits only	"12345"
isalnum()	letters + digits	"Python123".

* Session:7 String method:-

——— || islower and isupper || ———

1. What is purpose of the islower() method in python?

→ islower() checks whether all alphabetic characters in a string are lowercase.

ex:- text="Python"
print(text.islower()) # True.

2. When does the islower() method return True and When does it return false

→ - It return True when the string contains at least one character and all characters are lowercase.
- It return False When it contain an uppercase letter or has no character.

ex:- print("python".islower()) # True.
print("Python".islower()) # false.
print("123".islower()) # false
print("python123".islower()) # True.

3. What is the purpose of the is upper() methode?
→ isupper() checks whether all alphabetic character in a string are uppercase.
ex:- text = "PYTHON"
print(text.isupper())
# True.

4. What is difference ber islower() and isupper()?

islower()	isupper()
- checks lowercase	- checks uppercase.
- "python" → True	- "PYTHON" → True.
- "Python" → false	= "Python" → false.

• example:
print("hello".islower()) #True
print("HELLO".isupper()) # True.

——— || startswith() & endswith() || ———

5. What is the purpose of the startswith() method?
→ startswith() checks whether a string starts with specified substring.
- ex:- text = "Python Programming"
print(text.startswith("Python"))
# True.

Date
Page

6. What is the purpose of the endWith() method?

→ endswith() checks whether a string ends with a specified substring.

ex:- text = "Python programming"
print(text.endswith("programming"))

True.

find() and replace()

7. What does the find() method return when the given substring is found?

→ find() returns the index of the first occurrence of the substring.

ex:- text = "Hello Python"
print(text.find("python"))

6.

8. What does the find() method return when the given substring is not found?

→ It return -1.

text = "Hello Python"
print(text.find("Java"))

-1

9. What is the difference bern find() & replace()?

→

find()	replace()
- Searches for a substring	- change/replace a substring
- Return an index	- Return new string
- Not found -1	- if old text isn't found, string remains unchanged.

-example:

```
text = "I like python"
print(text.find("python"))
print(text.replace("python", "java")
#7
  I like java.
```

10. What is the purpose of the replace() method?

→ replace() is used to replace one substring with another substring.

```
ex:- text="I like python"
     new_text = text.replace("python", "java")
     print(new_text) # I like java.
```

11. Does the replace() method modify the original string directly? Why?

→ No,
strings are immutable, so replace() creates and returns a new string.

```
ex:- text="I like python"
     new_text=text.replace("python","java")
     print(text) # I like python
     print(new_text) # I like java.
```

Removing Whitespace

12. What is the purpose of the lstrip() method?

→ lstrip() removes whitespace form the left side (beginning) of string.

```
ex:- text="  python"
     print(text.lstrip()).
# python.
```

13. What is the purpose of the rstrip() method?

→ rstrip() removes whitespace from the right side (end) of a string.

ex:- text = "python "
print(text.rstrip())
# python.

14. What is the purpose of the strip() method?

→ strip() removes whitespace from both the beginning and the end of a string.

ex:- text = " python ".
print(text.strip()).
python.

15. What is the difference bern lstrip(), rstrip() and strip()

→

Method	Remove whitespace from
1) lstrip()	left side
2) rstrip()	Right side
3) strip()	Both side.

ex:- text = " python "
print(text.lstrip()) # "python "
print(text.rstrip()) # " python"
print(text.strip()) # "python"

(interview)

split() and partition()

16. What is the purpose of the split() method?

→ split() divides a string into multiple parts and returns them as list.

ex:- text = "python is easy"
result = text.split()
print(result).
['python', 'is', 'easy']

18. What happens when split() is used without specifying a separator?

→ When no separator is specified split() separates the string based on whitespace such as spaces, tabs & newlines.

19. What is the purpose of the partition() method?

→ partition() divides a string into three parts based on the first occurrence of a specified separator.
- It returns a tuple:
(before separator, separator, after separator).
- example:

```
text="python is easy"
result= text.partition("is")
print(result)
# ('python', 'is', 'easy')
```

V. IMP

20. What is the different bern split() and partition() :

split()	partition()
- Divides string into multiple parts	- Divides string into exactly 3 parts.
- Returns a list	- Return a tuple.
- separator is removed	- separator is kept.
- can split at multiple occurrences	- Uses the first occurrence
-ex:-	-ex:-
"a-b-c".split("-")	"a-b-c".partition("-")
→ ['a','b','c']	→ ('a','-','b-c')

Date .....
Page .....

Session 8: Case conversion methods:

1. What is the purpose of the lower() method in Python?

→ lower() converts all uppercase letters in a string into lowercase letters.

ex: text = "HELLO"
print(text.lower()) # hello

2. What is the purpose of the upper() method in Python?

→ upper() converts all lowercase letters in a string into uppercase letter.

ex:- text = "hello"
print(text.upper()) # HELLO.

3. What is the difference bet^n lower() & Upper() methode?

→ • lower() converts letters to lowercase.
• upper() converts letters to uppercase.
"HELLO".lower() # hello
"hello".upper() # HELLO.

4. What does the capitalize() method do?

→ capitalize() make the first character uppercase & converts the remaining characters to lowercase.

ex:- text = "hELLO WOrld"
print(text.capitalize())
# Hello world.

5. What does the tiHe() method do?

→ tiHe() converts the first letter of each Word to uppercase.

ex:- text = "hello python world"
print(text.tiHe()) # Hello Python World.

6. Difference bet^n capitalize() & title()

capitalize()	title().
- capitalizes the first character of the string	- capitalize the first character of each word
- ex "pratik shelke" → "Pratik shelke"	- e.x 'hello world" → "Hello World".

7. What is the purpose of swapcase()?

→ swapcase() changes uppercase letters to lowercase & lowercase letter to uppercase.

ex:- text = "Hello"
print(text.swapcase())
# hELLO.

8. How does swapcase() handle uppercase & lowercase characters?

→ It reverse the case of each alphabetic character

e.x:- "pythOn".swapcase() # pYtHoN.

Note:- Number & special characters remain unchanged.

9. Are string modified directly when we use lower(), upper() or swapcase()?

→ NO. python string are immutable, so these methods create & return a new string.

ex:- text = "HELLO"
new_text = text.lower()
print(text) # HELLO
print(new_text) # hello.

10. What is the purpose of the str() built-in function?

→ str() converts a value into it's string representation.

ex:- x=100
print(str(x)). # "100"

11. can an integer be converted into a string using str()?

→ Yes.

x=100
y=str(x)
print(type(y)). # <class 'str'>

12. can a float be converted into a string using str()?

→ Yes,

x=10.5
y=str(x)
print(type(y)). # <class 'str'>

13. Difference bet^n a built-in function and a string method in syntax?

→ -Built-in function:
ex. str(100)
- string method:
"HELLO".lower().
• Built-in function are called directly.
• string method are called using a string object followed by.

14. What is the purpose of the join() method?

→ join() combines multiple string into one string using a specified separator.

```
words = ["python", "is", "easy"]
result = " ".join(words)
print(result)  # python is easy.
```

15. How does the separator used with join() affect the resulting string?

→ The separator placed bet^n each string elemen

ex:-
```
words = ["A", "B", "C"]
print("-".join(words))  # A-B-C
print(" ".join(words))  # A B C
print(",".join(words))  # A,B,C
```

16. What type of element can be joined using join()

→ join() works with string elements.

ex:- `" ".join(["Hello", "World"])` # correct

- It does not directly join integers or floats.

`" ".join([1,2,3])` # TypeError

You must convert them to strings first.

17. What is the purpose of the split() method?

→ split() breaks a string into multiple parts & returns them as list.

ex:-
```
text = "python is easy"
print(text.split())
# ['python', 'is', 'easy']
```

18. What happens when split() is used without specifying a separator?

→ Without a separator, split() separates the string based on whitespace such as spaces, tabs & new lines.

ex:- text = "python is very easy"
print (text.split())
['python', 'is', 'very', 'easy']

19. Difference bet^n split() & join()?

split()	join()
- Breaks a string into parts	- combines strings into one string.
- Returns a list	- return a string
- string → list	- list → string.

20. Why is str() called a built-in function while lower(), upper() and join() are called string methods?

→ 1) str() is built-in function provided directly by python, so we call it like:
str(100)
2) lower(), upper(), and join() are methods associated with string objects, so they are called using the string and dot operator.
ex:- hello.upper()
" ".join(["Hello", "World"])

* Session 9 : Python console input/output.

1. What is console input and output in python?

→ 1) console input means taking data from the user through the keyboard while program is running.
- python uses the input() function for console input.
ex:- name = input("enter your name:")
print(name) # pratik.

Date
Page

2) console output :- console output means diplaying information on the screen/ console.
- python uses the print() function for console o/p.
- e.x :- Print ("Hello python").

2. Which function is used to accept input from the user in python?
→ The input() function is used to accept input from the user.
- e.x :- name = input ("enter your name")
print (name)

3. Which function is used display output on the console?
→ The print() function is used to display output on the console.
- e.x :- print ("Hello World").

4. What does the input() function return by default?
→ The input() function return the user's input as a string (str) by default.
- ex :- age = input("enter age:")
print (type (age))
If the user enters 20
<class 'str'>

5. Why is the data received from input() treated as a steing by default?
→ Because input() reads the user's keyboard input as text. python does not automatically assume whether the entered value is an integer, float or another type.

ex:- age = input("enter age:")
even if we enter 25, it is stored as "25"
- input() treats data as a string because user input is received as text.

6. How can we accept integer, float from the user using input()?
→ 1) use int() with input()
age = int(input("enter your age"))
print(age).
2) use float() with input()
price = float(input("enter price:"))
Print(price).

7. Why do we need type conversion when performing arithmetic operations on user input?
→ Because input() return a string by default & arithmetic operations such as addition require numeric data.

* Without conversion.	# With conversion:
a=input("enter value"):	a=int(input("entee value"))
b=input("enter value")	b=int(input("enter value"))
print(a+b)	print(a+b)
if input 10 & 20	# 30.
1020.	

8. What is the purpose of the int() function when working with user input?
→ int() is used to convert numeric text into a intege

9. What is the purpose of the float() function when working with user i/p?

→ float() is used convert numeric text into a floating-point value.

10. What is the purpose str() function in console I/P O/P ?

→ Str.() is used to convert a value into a string, especially when combining numers with text.

e.x :- age = 21
message = "My age is " + str(age)
print(message).

11. What is the purpose of the print() function in Python?

→ The print() function is used to display output on the console.

e.x :- name = "pratik"
print (name) # pratik.

12. What is the purpose of sep parameter in the print() function?

→ The sep (separator) parameter specifies what should be placed betn multiple value printed by print().

e.x :- print ("Python", "Java", "C++", sep="|")
Python | Java | C++.

13. What is the default value of the sep parameter in print().

→ The default value of sep is single space " "

e.x :- print ("Hello", "Python")
Hello Python.

14. What is the purpose of the 'end' parameter in the print() function?

→ The end parameter specifies what should be printed at the end of the output.
- by default print() move to a new line.

ex:- print("Hello" end=" ")
print("Python")
# Hello python.

Another example:-
print("Hello" end="***")
print("World")
# Hello*** World.

15. What is the default of the end parameter in print()?

→ The default value of end is newline character "\n".

ex:- print("Hello")
print('World')
# Hello
World.

16. What is formatted output in Python?

→ formatted output means displaying data in clear and organized format by combining text + variable/ value.

ex:- name = "pratik"
age = 23
print(f"My name is {name} and I am {age} year old.")
# my name is pratik and I am 23 year old.

17. What is an f-string + Why is it useful for

formatted output?

→ An f-string (formatted string literal) allows us to easily insert variable and expression directly inside a string using {}.

ex:-

```
a=10
b=20
print(f"sum = {a+b}")
# sum = 30.
```

* Python session 10: Decision control statement.

1. What are decision control statement in python?

→ Decision control statement used to make decision based on condition.

- Python mainly provides:
  - if
  - if-else
  - if-elif-else
  - Nested if.

- ex:-
```
age=20
if age>18:
    print("eligible").
```

2. Why do we use decision control statements in a program?

→ - We use them to control the flow of a program based on different conditions.
- Decision control statement allow a program to execute different blocks of code depending on conditions.

3. What is difference bet sequential execution & decision-based execution?

→

sequential execution	decision based execution
1) statement execute one after another	1) Execution depends on condition.
2) No decision is made	2) A decision is made.
3) follow normal sequence	3) may skip some block.
4) example: print() statement	4) example: if-else.

ex:- sequential execution

```
print("A")
print("B")
print("C")
```

ex:- age=20
if age >=18:
    print("Adult")

4. What is an if statement in python?

→ The if statement is used to execute a block of code when condition is True.

-syntax if condition:
    statement

-ex:- age=20
if age>=18:
    print("Adult").

5. When is the code inside an if block execute?

→ The code inside the if block is executed only when the condition is True.

6. Is the else block mandatory with an if statement?

→ No.

The else block is optional.

ex:- age=20
if age >=18:
    print("Adult").

Date
Page

7. What is the purpose of the else block?

→ The else block is executed when if condition is false.

ex:- age = 15
if age >= 18:
    print ("Adult")
else:
    print ("Minor").
Minor.

8. What is the difference bern if statement and if-else statement?

→

if	if-else.
1) Has only one condition block	1) Has two possible blocks.
2) execute code when condition is True.	2) Executes if or else.
3) Nothing happens if condition is false	3) else executes if condition is false.

9. What is an if-elif-else statement?

→ An if-elif-else statement is used when we have multiple conditions to check.

- syntax:-

```
if condition:
    statement
elif condition:
    statement
else:
    statement.
```

-ex:- marks=75

```
if marks >=90:
    print("A")
elif marks >=60:
    print("B")
else:
    print("C")
```

B.

10. Why do We use elif in Python?

→ elif means "else if".
It is used to check another condition when the previous if condition is false.

ex:-
```
marks=70
if mark >=90
    print("A")
elif marks >=60:
    print("B")
else
    print("C").
```

11. What is the important of indentation in python decision control statement?

→ Indentation tells Python. Which statement belong to a particular block.

ex:-
```
age=20
if age >=18:
    print("Adult")
    print("eligible")
```

Here, both print() statement belong to the if block because they are indented.

Indentation

12. What happens if the condition of statement inside an if block is incorrect?

→ Python may produce IndentationError or SyntaxError depending on the mistake.

13. What is a condition in python?

→ A condition is an expression that is evaluated as True or false.

e.x':- age = 20
age >= 18 # True.
age < 18 # false.

- conditions are commonly created using comparison operators.

14. What type of result does a comparison condition produce?

→ A comparison condition produce a Boolean (bool) value.

- True.
- false.

e.x':- 10>5 # True
10<5 # false.

15. What is difference bet^n the = & == operators?

→

=	==
1) Assignment operator	1) equality comparison operator.
2) Assign the value to variable.	2) checks whether two values are equal.
3) ex :- x=10	3) ex :- x==10

16. What is the difference bet? >, <, >= and <= comparis -on operator?

→

operator	Meaning	example	Result.
>	Greater than	10>5	True
<	Less than	5<10	True
>=	Greater than or equal	10>=10	True
<=	less than or equal to	5<=10	True.

17. How is the execution flow different bet^n if, if-else and if-elif-else?

→ if → executes block only if condition is true.
if-else → executes either if or else.
if-elif-else → checks multiple conditions and executes the first matching block.

* session 11: operators:

1. What are logical operator in python?

→ logical operator are used to combine multiple conditions and produce a Boolean result (true or false)

- python logical operators:
  - and
  - or
  - not.

ex:- age=20
print.(a>=18 and age<=60)
# True.

- logical operators are used to combine or reverse conditions in python:

2. What is purpose of the "and" operator in python?

→ The and operator is used when all condition must be true.

ex:- age=20
if age>=18 and age<=60:
print("eligible")
# eligible.

-and → All condition should be true.

3. What does the "and" operator return True?

→

A	B	and
True	True	True
True	false	false
False	True	false
false	false	false.

ex:- print(True and True). # True.

4. What is the purpose of "or" operator in python?

→ The "or" operator is used when at least one condition should be true.

ex:- day="sunday"
if day=="saturday" or day=="sunday":
print("Weekend")
# Weekend.

5. When does the "or" operator return True?

→ "or" return True when at least on condition is True.

A	B	or
True	True	True
True	false	True
false	True	True
false	false	false

ex:- print (True or false)
1 # True

6. What is the difference bern the 'and' and 'or' operators?

→

and	or
1> Requires all condition to be true	1> Requires at least one condition to be true.
2> one false condition makes the result false	2>. on true condition makes the result true.
3> Similar to AND logic	3>. similar OR. Logic

7. What will 'and' operator return if both conditions are false?

→ it return false:
ex:- print (false and false) # false.

8. What will the "or" operator return if both conditions are false?

→ It returns false:
print (false or false)
false.
- for "or", even one true condition would make the result true.

Conditional Expression

9. What is a conditional expression in python?

→ A conditional expression is a short way of writting an if-else statement in one line.
ex:-

```
age = 20
result = "Adult" if age >= 18 else "Minor"
print(result)
 # Adult.
```

10. Why is a conditional expression also called a one-line if-else?

→ Because the complete if-else decision is written in single line.

- Normal if:else:

```
if age >= 18:
    result = "Adult"
else:
    result = "Minor"
```

- conditional expression:

```
result = "Adult" if age >= 18 else "Minor".
```

11. What is the syntax of a conditional expression in python?

→ - syntax:-

```
value_if_true if condition else value_is_false
```

- ex:-

```
result = "Adult" if age >= 18 else "Minor".
```

12. How is a conditional expression different from a normal if-else statement?

→

conditional expression	Normal if else
1) usually written in one line	1) usually written in multiple lines.
2) short & concise	2) More detailed.
3) useful for simple decision	3) better for complex logic.

4) Return / selects a value

4) can executes multiple statement.

|| all() function ||

13. What is the purpose of the all() built-in function?

→ The all() function checks whether all elements in an iterable are true.

-ex:- values = [True, True, True]
print (all (value))
True.

- An iterable can be a list, tuple, set, etc.

14. When does all() return True?

→ all() return True when every element is true

ex:- number = [True, True, True]
print (all (numbers))
True.

② It also works with conditions:

ex:- numbers = [10, 20, 30]
print (all (x > 0 for x in numbers))
True.

15. When does all() return false?

→ all() return false when at least one element is false.

-ex:- values = [True, false, True]
print (all (values))
false.

16. How is all() related to the logical and operator?

→ Both require all conditions to be True.

1) Logical and:-

```
print (True and True and True)
# True.
```

2) all()

```
print( all ([True, False, True])
# False.
```

- all() Works like an AND operation across all elements of an iterable.

any() function

17. What is the purpose of the 'any()' built-in function?

→ The any() function checks whether at least one element in an iterable is true.

ex:-
```
value=[false, True, false]
print (any (value))
# True.
```

18. When does any() return True?

→ any() return True When at least on element is True.

19. When does any() return false?

→ any() return false When all elements are False.

- value=[false, false, false]
```
print (any (value))
# false.
```

20. How is any() related to the logical or operator?

→ Both return True When at least one condition is True.

1] logical or :-

```
print (False or True or False)
# True.
```

2] any () :-

```
print (any ([false, True, false]))
# True.
```

* session 12: Repetition control instruction (While, for loop

1. What is repetition control instruction in python?

→ Repetition control instruction are used to execute a block of code repeatedly based on a condition or for each item in a sequence.

- python mainly provides:
  - While loop
  - for loop
- It also provide loop control statement:
  - break
  - continue
  - else with loop.

2. Why do we use loops in python?

→ We use loops to avoid writing the same code repeatedly.

- without loop:

```
print("Hello")
print("Hello")
print("Hello")
```

- With loop:

```
for i in range(3):
    print("Hello")
```

3. What is While loop & When is it used?

→ A While loop repeatedly executes a block of code as long as it's condition is True.

- Syntax:

```
While condition:
    statement.
```

- ex:-

```
count = 1
While count <= 3:
    print(count)
    count = count + 1
```

4. How does a While loop decide Whether to continue it's execution?

→ The While loop checks it's condition before every iteration.

- condition True: loop executes.
- condition false: loop stops.

**5. Why is updating the loop-control variable important in While loop?

→ The loop-control variable must usually be updated so that the condition eventually become false.

- eventually ⇒ after some time.

ex:-

```
count = 1
While count <= 3:
    print(count)
    count += 3
```

Here count += 3 changes the value.

1 → 2 → 3 → 4

When count becomes 4

4 <= 3 → false. stop the loop.

6. What happens if the condition of a while loop never become false?

→ If the condition never becomes false, the while loop becomes an infinite loop.

-e.x':-

```
count=1
While count <=3:
    print(count).
```

7. What is the purpose of else block with while loop?

→ The else block is used to execute code after the while loop finishes normally.

-e.x':-

```
count=1
While count <=3:
    print (count)
    count+=3
else:
    print ("loop completed")
```

1
2
3
loop completed.

8. When does the else block of a while loop execute?

→ The else block execute when the while loop terminates normally because it's condition becomes false.

9. What happens to the else block if the while loop is terminated using break?

→ If the loop execute terminated using break, the else block does not executes.

e.x':- count=1

```
while count<=5:
    if count==3:
        break
```

```
        print(count)                    # 1
        count += 1                      2
    else:
        print("loop completed").
```

for loop

10. What is a for loop in python?

→ A for loop is used to iterate over the elements of a sequence or iterable, one item at a time.

- example:- for i in [10, 20, 30]:
  print(i).

- A for loop is used to executes a block of code for each element of an iterable.

11. What is the difference bet'n a while loop and for loop

→

While loop	for loop
1) Based mainly on a condition	1) Iterates over an iterable.
2) Useful when repetition depends on a condition	2) Useful when iterating through known items/range
3) programmer usually manages the loop-control variable.	3) loop variable automatically assigned each iteration
4) ex:- while age<18	4) ex:- for x in range(5):

12 What is sequence in python? Give example of sequences used with a for loop.

A sequence is an ordered collection of elements.

- common example includes:
  - steing
  - list
  - tuple
  - range.

-ex:- for item in [10,20,30]:
print(item).

13. Can a for loop iterate through the characters of a string? How?

→ Yes, A for loop can iterate through each character of a string.

-ex:- name="pratik"
for i in name:
print(i).

14. can foe loop iterate through the element of list & tuple?

→ Yes,

1) list:- numbers=[10,20,30] # 10 20 30
for num in number:
print(num)

2) Tuple:- numbers=(10,20,30) # 10 20 30
for num in numbers:
print(num)

range() function

15. What is the purpose of range() function?

→ The range() function generates a sequence of number, commonly use with for loop.

-ex:- for i in range(5):
print(i).

16. What are the start, stop & step parameters of range?

→

→ Syntax:- range(start, stop, step).

ex:- for i in range(2,10,2)
    print(i).

2
4
6
8

17. Is the stop value included when using range?

→ No,
The stop value is excluded.

-for i in range(1,5):
    print(i)

1
2
3
4

——— // Loop control statement // ———

18. What is the purpose of break statement?

→ the .break statement is used to immediately terminate a loop.

-ex:- for i in range(1,6):
    if i == 3:
        break
    print(i)

1
2

19. What is the purpose of continue statement?

→ The continue statement is used to skip the current iteration & move to the next iteration of the loop.

-ex:- for i in range(1,6):
    if i==3:
        continue
    print(i).

1
2
4
5.

20. What is the difference bet^n break & continue?

break	continue
1) Terminates the entire loop	1) skips only current iteration.
2) loop stop immediately	2) loop continues.
3) used to exit a loop	3) used to skip an iteration

* Session 13: list, Indexing, slicing & Iteration:

1. What is a list in python?

→ A list is ordered & mutable collection of element in python.
- It can store multiple values in a single variable.
- numbers = [10,20,30,40]
  print(numbers)

2. Why is a list called a container data type?

→ A list is called a container data type because it can contain multiple elements inside a single object.
ex:- student=["Pratik", 21, "MCA"]

3. How is a list represented in python?

→ A list is represented using square brackets [].
ex:- numbers = [10,20,30,40].

4. can a python list contain element of different data type?

→ Yes, python lists can contain element of different data type.

ex:- data = [10, "python", 3.14, True]

- This called a heterogenous list.

5. What is difference bet^n homogenous list & heterogenous list.

→ * Homogenous list:- A list containing element of the same data type.

ex:- list = [10, 20, 30, 40]

All element are integer.

* Heterogeneous list: A list containing element of different data type.

ex:- data = [10, "python", 3.14, True]

6. Are duplicate elements allowed in a python list?

→ Yes, Lists allows duplicate elements.

ex:- number = [10, 20, 10, 30, 40, 20]

print (number).

7. What happens when the * operator is used with a list?

→ The * operator repeats the list multiple times.

ex:- numbers = [1, 2, 3]

print (numbers * 3)

[1, 2, 3, 1, 2, 3, 1, 2, 3]

List Indexing

8. What is indexing in a python list?

→ Indexing means accessing an individual element of a list using it position or index.

ex:- numbers = [10,20,30,40]
print (number[0]) # 10

9. What is difference bet^n positive indexing & negative indexing.

→

positive indexing	Negative indexing.
1) starts from 0	1) starts from -1.
2) Moves from left to right	2) Move from right to left.
3) 0,1,2,3....	3) -1,-2,-3.....

ex:- numbers = [10,20,30,40)

element	10	20	30	40
positive	0	1	2	3
Negative	-4	-3	-2	-1

10. positive indexing starts from 0.
ex:- print (number[0]) # 10

11. Negative indexing start from -1
ex:- print (number[-1]) # 40.

12. What happens if We try to access a list element using an index that does not exist?

→ Accessing a non-existing list index raises an IndexError.

List Slicing

13. What is list slicing?

→ list slicing is used to extract a portion of a list.

ex:- numbers=[10,20,30,40,50]
print(numbers[1:4])
# [20, 30, 40]

14. What is the general systax for slicing a list?

→ The general syntax is:

list[start:stop:step]

ex:- print(numbers[1:5:2])
[20,40]

15. Is the stop index included in a list slicing?

→ No,
The stop index is always excluded.

16. What are the default value of start, stop & step in list slicing?

→
- start → beginning of the list
- stop → end of the list
- step → 1

ex:- number=[10,20,30,40]
print(number[:])
#[10,20,30,40]

17. How can we iterate through a list using a for loop?

→ We can use a for loop to access each element one by one.

ex:- numbers = [10, 20, 30, 40]
for num in numbers:
print (num).

18. How can we iterate through a list using while loop?

→ We can use an indexing variable with a while loop.

ex:- numbers = [10, 20, 30, 40]
i = 0
while i <= len(numbers):
print (numbers[i])
i += 1

19. What is the purpose of the enumerate() function wh en iterating over a list?

→ enumerate() is used when we need both the index & the element while iterating.

- example:

names = ["Amit", "Rahul", "pratik"]
for index, name in enumerate(names):
Print (index, name).

0 Amit
1 Rahul
2 pratik.

- You can also specify starting index:

for index, name in enumerate (names, start = 1):
print (index, name)

1 Amit
2 Rahul
3 pratik.

Date
Page

* Session 14: list properties

1. What does mutability mean in python?

→ mutability means that an object's contents can be changed after the object is created.

-example: num=[10,20,30]

```
num[1] = 50
print(num)
# [10,50,30]
```

2. Why is a list called a mutable data type?

→ A list is called mutable because we can change, add or remove elements after creating the list.

ex:-
```
num=[10,20,30]
num[0] =100
num.append(40)
print(num)   # [100,20,30,40]
```

3. How can we add a new element to an existing list?

→ We can use method such as:

- append()
- insert()
- extend()

e.x:-
```
num=[10,20,30]
num.append(40)
print(num)   # [10,20,30,40]
```

4. How can We remove an element from a list?

→ We can use methods such as:

- remove()
- pop()

Date
Page

- clear()

ex:- num=[10,20,30,40]
num.remove(20)
print(num) # [10,30,40]

remove() removes an elements based on it's value.

5. How can we replace an existing element in a list?

→ We can replace an element using it's index.-

ex:- num=[10,20,30]
num[1]=50
print(num) # [10,50,30]

- because lists are mutable, we can replace an element by assigning a new value to it's index.

——— || list concatenation. || ———

6. What is list concatenation?

→ list concatenation means joining two or more lists to create a new list.

ex:- list1=[10,20]
list2=[30,40]
result=list1+list2
print(result) # [10,20,30,40]

7. Which operator is used to concatenate two list?

→ The "+" operator is used to concatenate two list.

——— || list function || ———

8. What is the purpose of list() function?

→ The list() function is used to create a list or convert an iterable into a list.

-ex:- num= list ((10,20,30))
print (num)
[10,20,30]

9. Which python data types can be converted into a list using list()?

→ list() can convert iterable objects into a list.
- common example:
  - string
  - Tuple
  - Set
  - Dictionary
  - Range.
  - other iterables.

10. What happens when a string is converted into list using list()?

→ The string is converted into a list of individual character.
-ex:- text = "python"
result= list (text).
print (result) = ['p', 'y', 't', 'h', 'o', 'n']

11. What happens when a dictionary is converted into a list using list()?

→ By default, list() converts a dictionary into list of it's keys.
ex:- student = { "name": "pratik",
"age": 21
}
result = list (student)
print (result)
["name', 'age']

Date
Page

* Important :-
list (dictionary) → give keys, not value.
list (student.values()) → for value.
list (student.items()) → for key - values.

12. What happens when a range object is converted into a list?

→ The values generated by the range() object are converted into a list.

ex :- num = list (range (1,6))
print (num)
[1,2,3,4,5]

Imp —— Aliasing & cloning ——

13. What is aliasing in python lists?

→ Aliasing means two or more variable refer to the same list object.

ex :- list1 = [10,20,30]
list2 = list1
list2.append(40)
print (list1) # [10,20,30,40]
print (list2) # [10,20,30,40]
The change is visible through both variable.

14. What happens when two variables are assigned to the same list using assignment?

→ Both variable refer to the same list object.

a = [10,20,30]
b = a
b[0] = 100
print (a) [100,20,30]
print (b) [100,20,30]

15. What is the purpose of the id() function when discussing lists?

→ The id() function return an integer that identifies an object during it's lifetime.
- It can help us check whether two variables refer to the same object.
- e.x':-
```
a = [10,20]
b = a
print(id(a))
print(id(b))
```

cloning / copying

16. What is cloning of a list?

→ cloning means creating a separate copy of an existing list.
- e.x':-
```
a = [10,20,30]
b = a.copy()
```
Now a and b are separate list objects.

17. How can we create a separate copy of a list using the copy() method?

→ Use the list's copy() method.
```
a = [10,20,30]
b = a.copy()
b[0] = 100
print(a)   # [10,20,30]
print(b)   # [100,20,30]
```
- the original list a is not changed.

* Important:
```
b = a.copy()
```
→ creates a shallow copy of the list.

Date
Page

18. What is the difference bet^n aliasing & cloning?

→

Aliasing	cloning.
1) Both variables refer to the same list	1) creates a separate list.
2) No New list is created	2) New list object is created.
3) changes affect both variables.	3) changes to the copied list don't affect the original for the list's own element
4) b=a	4) b=a.copy()
5) same id()	5) Different id()

——— Membership & Identity ———

19. What is the purpose of the in and not in operators when working with lists?

→ They are membership operators used to check whether an element exists in a list.

*in:-
Returns True if the element exists.

```
num = [10,20,30]
print (20 in num) # True.
```

*not in:- Return True if the element does not exist.

```
print (50 not in num) # True.
```

IMP

20. What is the difference bern 'is' is 'operator' and the == operator When Working With lists?

→ * == → compares values / content.
- It checks Whether two lists contain equal elements.

e.x':- a = [10, 20, 30]
b = [10, 20, 30]
print (a == b) # True.

The contents are the same.

* is → compares object identity.
It checks Whether both Variables refer to the same object.
- print (a is b)
false.
Because a & b are separate list objects.

* e.x':- a = [10, 20, 30]
b = a
print (a == b) # True
print (a is b) # True.

* == → Same value?
is → same object?

* Int Session 15: Built in function & methods.

Bmg
1. What is the difference bern a built-in function & list method?

→ 1) Built-in function: A built in function is predefined function provided by python that can be used independently.

- example:-

```
len(my_list)
max(my_list)
min(my_list)
sum(my_list)
```

2) list Method:- A list method is function that the belong to a list object and called using.

ex:-
```
my_list.append(10)
my_list.remove(20)
my_list.sort()
```

2. What is the syntax for calling a built-in function on a list?

→ syntax:-

```
function_name(list_name)
```

ex:-
```
num = [10,20,30]
print(len(num)) #3
```

- A built-in function is called by passing the list as an argument.

3. What is the syntax for calling a list method?

→ syntax:-

```
list_name.methode_name()
```

example:
```
num=[10,20,30]
num.append(40)
print(num)  #[10,20,30,40]
```

4. What is the purpose of len() when used with a list?

→ len() return the number of element present in the list.

e.x':- num = [10, 20, 30, 40]
print (len (num)) #4

5. What is the purpose of max() & min()?

→ max() → return the largest value.
min() → return the smallest value.
e.x':- num = [10, 50, 20, 5, 30]
print (max (num)) # 50
print (min (num)) # 5

6. What is the purpose of Sum()?

→ Sum() calculates the total of numeric element in a list.
e.x':- num = [10, 20, 30]
print (Sum (num)) # 60

7. What does any() return when at least one elemen is truthy?

→ any() return "True" if at least one element is truthy
e.x':- num = [0, 0, 5, 0]
print (any (num)) # True.

8. What does all() return when all element are truthy?

→ all() return "True" when every element is truthy.
e.x':- num = [1, 2, 3, 4]
print (all (num))
True.

9. Difference betn any() & all()

IMP

any()
1) check if at least one is truthy
2) one truthy value is enough
3] any([0,0,1]) → True

all()
1) checks if all are truthy.
2) every value must be truthy.
3) all([1,1,1]) → True.

10. What is the purpose of sorted()?
→ sorted() return a new sorted list.
ex:- num = [30,10,20]
new_list = sorted(num)
print(new_list) # [10,20,30]
- sorted() return a new sorted list without modifying the original list.

11. What is the purposed of reversed()?
→ reversed() return an iterator that provides the element in reverse order.
- reversed(iter)
ex:- num = [10,20,30]
result = reversed(num)
print(list(result))
# [30,20,10]
- important: print(reversed(num)) does not directly print a normal list.

IMP 12. Difference betn sorted() & sort():
1) sorted:-
- built-in function.
- return the new list
- original list remains unchanged.-

ex:- num = [30, 10, 20]
result = sorted(num)
print(result) # [10, 20, 30]
print(num) # [30, 10, 20]

2) sort():
- list method
- changes the original list
- return None

ex:- num = [30, 10, 20]
num.sort()
print(num) # [10, 20, 30]

13: What is the purpose of append()?
→ append() adds a single element at the end of a list.

ex:- num = [10, 20, 30]
num.append(40)
print(num) # [10, 20, 30, 40]

Imp
14. Difference ber append() & extend?

append()	extend()
1) Adds one element	1) Add multiple element
2) [1,2].append([3,4])	2) [1,2].extend([3,4])
3) # [1, 2, [3, 4]]	3) # [1, 2, 3, 4]

15. What is the purpose of insert?
→ insert() adds an element at a specified position in a list.
- syntax: list.insert(index, value)

Date .....
Page .....

-ex:- num=[10,20,30]
num.insert(1,15)
print(num) # [10,15,20,30].

16. Difference betn remove() & pop()

→

remove()	pop()
1) Removes by value	1) Remove by index
2) list.remove(value)	2) list.pop(index)
3) Return None	3) Return removed element.
4) ex:- remove(20)	4) pop(1).

ex:- num=[10,20,30]
num.remove(20)
print(num) # [10,30]

ex:- num=[10,20,30]
x=num.pop(1)
print(x) =20
print(num)=[10,30]

17. What is the purpose of index()?

→ index() return the index of the ~~1st~~ first occurrence of specified element.

ex:- num=[10,20,30,20]
print(num.index(20)) # 1.

18. What is the purpose of count()?

→ count() return the number of occurrences of a specified element.

ex:- num=[10,20,30,20,30]
print(num.count(20))
# 3.

19. Difference bet sort() & reverse():

→ 1) sort(): change the order according to sorting.
2) reverse():- Simply reverse the current order.

```
ex:- num=[30,10,20]
     num.sort()
     print(num)   # [10,20,30]

ex:- num=[10,20,30]
     num.reverse()
     print(num) # [30,20,10]
```

20. What is purpose of copy()?

→ copy() creates a separate shallow copy of a list. It is useful when we want to modify new list without modifying the original list.

ex:-

```
list1=[10,20,30]
list2=list1.copy()
list2.append(40)
print(list1)  # [10,20,30]
print(list2)  # [10,20,30,40]
```

Date
Page

* Session 16: Tuple:

1. What is a tuple in python?
→ A tuple is an ordered collection of element in Python.
- It is immutable, which means we cannot modify it's elements after creation.
ex:- number=(10,20,30,40)
print(number) # (10,20,30,40)

2. How do we create a tuple in python?
→ We normally create a tuple using parentheses ().
- example: num=(10,20,30)
names=("pratik", "Rahul", "Amit")
We can also use the tuple() constructor.
num=tuple([10,20,30])
print(num) # (10,20,30)

3. What is the different betn a homogeneous tuple and a heterogeneous tuple?
→ - A homogenous tuple contains elements of same data type
ex:- num=(10,20,30,40)
- A heterogeneous tuple contains elements of different data type.
ex:- student=("pratik", 22, 9.5, True)

4. Can a tuple contain duplicate element?
→ Yes, Tuple allow duplicate elements.
num=(10,20,10,30,20)
print(num)
# (10,20,10,30,20)

5. How do we create empty tuple?

→ Use empty parentheses:-

```
t=()
print(t)
print(type(t))
# ()
  <class 'tuple'>
```

6. How do we create a tuple containing only one element?

→ We must use a comma ,

e.x:-
```
t=(10,)
print(t)
print(type(t))
# (10,)
  <class 'tuple'>
```

7. What is the difference betn (10) & (10,) ?

→
```
a=(10)
print(type(a))    # <class 'int'>
a=(10,)
print(type(b))    # <class 'tuple'>
```

8. What is tuple indexing, & what is the difference betn positive & negative indexing?

→ Indexing is used to access an individual element from a tuple.

e.x:- num = (10,20,30,40,50)

positive indexing :
start from: 0

Negative indexing :
start from: -1 from right to left

Date
Page

9. What is tuple slicing, & What is the general syntax?

→ => slicing is used to extract a portion of a tuple.

- Syntax :- tuple [start : stop : step]

ex :-

```
num = (10,20,30,40,50)
print (num [1:4])  # (20,30,40)
```

* Negative indexing.

```
print (num [-4:-1])  # (20,30,40)
```

10. How can We iterate through a tuple using for loop?

→ We can directly access each element using a for loop.

ex :-
```
num = (10,20,30)
for nums in num:
    print (nums)
```

11. How can We iterate through a tuple using While loop?

→
```
num = (10,20,30)
i = 0
While i < len (num)
    print (num[i])
    i += 1.
```

12. What is purpose of enumerate() When iterating through a tuple?

→ enumerate() gives us both index & value while iterating

ex :-
```
num = (10,20,30)
for index, value in enumerate (num):
    print (index, value)
```

#
```
0 10
1 20
2 30.
```

13. Why is a tuple called immutable data type?

→ A tuple is called immutable because it's elements cannot be changed after creation.

- ex:- num = (10,20,30)
  num[0] = 100
  # TypeError: 'tuple' object does not support item assignment.

Note:- If a tuple contain a mutable object such as a list that nested object itself can still be modified.

ex:- t = ([10,20],30)
  t[0].append(40)
  print(t) # ([10,20,40],30)

14. concatenation means joining two or more tuple and + operator is used.

ex:- t1 = (10,20)
  t2 = (30,40)
  result = t1 + t2
  print(result) # (10,20,30,40)

Important: this create a new list.

15. How can different data type be converted into tuple?

→ The tuple() function can convert several iterable object into a tuple.

1) string → Tuple.
  text = "python"
  print(tuple(text))

2) list → tuple
  num = [10,20,30]
  Print(tuple(num))

16. What is aliasing in tuple?

→ Aliasing means two variables refer to the same tuple object.

```
e.x:- t1=(10,20,30)
      t2= t1
      print (t2 is t1)
   # True.
```

17 What is the difference betn in & is when working with tuple?

→ 1) in: check whether a value exists inside the tuple.

```
e.x:- num=(10,20,30)
      print (20 in num) # True.
```

2) is: check whether two variable refer to the same object.

```
t1=(10,20)
t2=t1
print (t1 is t2) # True.
```

18. How does python compare two tuple?

→ python compares tuples element by element from left or right.

```
e.x:
  t1=(10,20,30)
  t2=(10,20,40)
  print (t1<t2) # True.
```

② print ((5,100) < (10,1)) # True.

because python first compares 5 & 10. It does not need compare 100 & 1.

19. How can we check whether a tuple is empty using not?

→ not tuple_name is a simple pythonic way to check whether a tuple is empty.

e.x':-

```
t=(10,20)
if not t:
    print("Tuple is empty")
else:
    print("Tuple is not empty")
```

Tuple is not empty.

20 Which built-in functions can be used with tuple?

→ len(), max(), min(), sum(), any(), all(), sorted(), reversed().

* Session 17: Set.

1. What is a set in python?

→ A set is a built-in data type in python used to store multiple unique element.
- sets do not allow duplicate element.
- sets are unordered
- sets are mutable.
- sets are written using {} with elements inside.

e.x':-
```
num={10,20,30,40}
print(num)
```

2. How do we create a set in python?

→ We can create a set using curly braces {} with element.

num = {10, 20, 30}

We can also use the set() constructor.

num = set ([10, 20, 30])

3. What is the difference betn a set and a list with respect to duplicate element?

list	set
1) Allows duplicates	1) Does not allow duplicates.
2) [10, 20, 10]	2) {10, 20, 10} becomes {10, 20}
3) Maintains insertion order	3) Does not guarantee order.

4. Can a set contain homogeneous as well as heterogeneous element:

→ Yes,

1) Homogeneous elements: same data type:

S = {10, 20, 30}

2) Heterogeneous elements: different data type:

S = {10, "Python", 3.5}

5. Why are duplicate element not stored in a set?

→ Sets use hashing to identify their elements.

- When we add an element, python checks whether an equivalent element already exists in the set.

S = {10, 20, 30, 10}

print(S)

5. What is hashing in the context of a python set?

→ Hashing is a process of converting a hashable object into a fixed hash value using it's hash() value.

- Python uses this hash value to efficiently stored & find elements in a set.

ex:- x=10
print(hash(x))

7. How can we create an empty set in python?

→ Use the set() function.

ex:- s=set()
print(s)
# set()

8. Why does {} create an empty dictionary instead of an empty set?

→ python uses {} for an empty dictionary.

ex:- x={}
print(type(x)
# <class 'dict'>

- To create an empty set:

x=set()
print(type(x)) # <class 'set'>

9. Why is a set called an unordered collection?

→ A set does not provide a guaranteed positional order for it's element.

-for example:-

s={10,20,30,40}

We should not depend on the displayed order of set elements.

10. Why can we not use indexing & slicing with a set?

→ Sets do not have position like lists & strings.

Therefore, indexing & slicing are not supported.

```
S={10,20,30}
print(S[0])
# TypeError.
```

11. How can we iterate through the element of a set?

→ We can use a for loop.

```
S={10,20,30,40}
for value in S:
    print(value)
```

12. Why is a set called a mutable data type?

→ A set is mutable because we can change it's contents after creation.

ex:-
```
S={10,20,30}
S.add(40)
print(S)
```

We also remove elements:

```
S.remove(20)
```

13.

Aliasing	cloning
1) S2=S1	1) S2=S1.copy()
2) Same object	2) Different objects.
3) change affect both	3) change remain in separate.

* Session:18: Set method:

1. What is the difference bet^n add() & update() in a set?

→ • add() is used to add one element to a set.

• update() is used to add multiple elements from an iterable such as a list, tuple, set etc.

```
s={10,20}
s.add(30)
print(s)   {10,20,30}

s.update([40,50])
print(s)
```

2. What happen if you add an element that already exists in aset?

→ If an element already exists, add() does not create a duplicate because sets contain only unique element

```
e.x:- s={10,20,30}
      s.add(20)
      print(s)  ={10,20,30}
```

3. What is difference bet remove() & discard()?

→

remove()	discard()
1) Removes an element from a set	1) Removes an element from a set.
2) Raises KeyError if element is absent	2) Does not raise an error if element is absent.
e.x:- s={10,20,30} s.remove(20) print(s) # {10,30}	e.x:- s={10,20,30} s.discard(50) print(s) {10,20,30}

4. Which error is raised the remove() is used on an element that is not present?

→ remove() raised a key Error if the specified element does not exist in the set.

5. What do the methods add(), remove() & clear(), return?

→ All three method return None.
- They modify the original set but do not return the modified set.

6. Why does a set not print elements in the order in which they were added?

→ A set is unordered, so we cannot rely on the order in which it's element are stored or displayed.

7. After using clear(), Why does python print set() instead of {}?

→ python uses set() to represent an empty set because {} is reserved for an empty dictionary.

ex:- s={10,20,30}.
s.clear()
print(s)
# set(). # empty set.
={} # empty dictionary.

8. What is the difference bet^n set2=set1 & set2=set1.copy()?

→ set2=set1 create another reference to the same set, whereas set2 = set1.copy() create a separate copy of the set.

ex:- set1={10,20,30}
set2=set1
set1.add(40)
print(set1)
print(set2)
# {10,20,30,40}
{10,20,30,40}

ex 2: set1={10,20,30}
set2=set1.copy()
set1.~~copy~~ add(40)
print(set1)
print(set2)
# {10,20,30,40}
{10,20,30}

Imp

9. What is the difference bet^n s.add("hello") & s.update("hello")?

→ add("hello") adds "hello" as one element, while update("hello") treats the string as an iterable & add it's individual character.

add("hello")
↓
one element
"hello"

update("hello")
↓
characters
{h, e, l, l, o}

10. can We add a list to set using add()? can we add a tuple? explain Why.

→ list using add()
No,
s={10,20}
s.add([30,40])
This gives: TypeError: unhashable type: 'list'

Tuple using add():
Yes, provided the tuple's contents are hashable:
s={10,20}
s.add((30,40))
print(s)
# {10,20,(30,40)}

- We cannot add a list to a set because lists are mutable & unhashable. we can add a tuple if all it's element are hashable because tuple can be hashable.

Date
Page

* Session 18: Dictionary:

1. What is dictionary in python? How is different from a list?

→ A Dictionary is mutable data type that stores data in key-value pairs.

-ex:- student = { "name" : "pratik",
"age" : 22,
"course" : "MCA"
}

* Dictionary	list
1) Stored key-value pair	1) Stored individual element.
2) Accessed using keys	2) Accessed using index.
3) written using {}	3) written using []
4) key must be unique	4) Duplicate element are allow
5) ex:- d["name"]	5) Ex: l[0]

2. How are key-value pair written in a dictionary?

→ A key-value pair is written using a colon :

Syntax:

```
dictionary = {
    key : value
}
```

ex:- dict1 = { "name" : "pratik",
"age" : 22
}

3. Can two different keys have the same value?

→ Yes,
Different key can have the same value.

ex:- student = {
"name": "pratik",
"nickname": "pratik",
"city": "pune"
}
print (student)

4. Can a dictionary have duplicate keys?

→ No,
A dictionary cannot stored duplicate keys.
- If the same key is written more than once, Python keeps the last value.
- example:

student = {
"name": "pratik",
"age": 22,
"name": "om"
}
print (student)
{'name': 'om', 'age': 22}

5. How do you access a value from a dictionary? What happens if the key does not exists?

→ We normally access a value using keys.
ex:- student = {
"name": "pratik",
"age": 22
}
print (student["name"])
pratik
- If key does not exist, dict[key] raise keyError, while get() return None by default

6. Why do indexing (dict1[0]) + slicing (dict1[::]) not Work in a dictionary?

→ Dictionary elements are accessed using keys rather position, so normal numerical indexing & slicing are not support.

7. What is the difference betn items(), keys() & value() when looping through a dictionary.

→ consider,

```
student={ "name": "pratik",
          "age":22,
          "city": "pune"
        }
```

1) keys(): Return the keys.

```
for key in student.keys():
    print(key)
```

2) values(): Return the values.

```
for value in student.values():
    print(value)
```

3) items() Return both key & value.

```
for key, value in student.items():
    print(key, value).
```

8. How can you get the index position along with the keys & value while looping?

→ use the enumerate() function with items().

```
for index,(key, value) in enumerate(student.items()):
    print(index ,key, value).
```

9. Is a dictionary mutable or immutable?

→ A dictionary is mutable.

10. How do you add a new key-value pair to an existing dictionary.

→ - we can use:

dictionary[key] = value.

- example:

```
student = {
    "name": "pratik"
    "age": 22
}
student["city"] = "pune"
print(student)
```

* Session 19: Dictionary (2).

1) How can you convert a tuple or a list into a dictionary? What structure must the tuple or list have?

→ We can the dict() function to convert a list of tuple a dictionary.

- The list / tuple must contain key-value pairs, where each pair has exactly 2 element.

```
- data = [("name", "pratik"), ("age", 22)]
  d = dict(data)
  print(d)
  # {'name': 'pratik', 'age': 22}.
```

2) How can you create a dictionary from set of elements where every key gets the same default value?

→ We can use dict.fromkeys().

```
keys = {"a", "b", "C"}
d = dict.fromkeys(keys, 0)
print(d).
# {'a':0, 'b':0, 'c':0}
```

Here, every key gets the same default value 0.

3. How can you create a dictionary using range() as keys and a default value for every key?

→ We can use dict.fromkeys(range(....), value) to create dictionary keys from a range & assign the same default value to all keys.

-ex:-

```
d=dict.fromkeys(range(1,5),"python")
print(d)
```

4. What is aliasing in dictionary?

→ Aliasing means two variable refer to the same dictionary object.

```
dict1 = {'0': 10, 'b': 20}
dict2 = dict1
dict2["a"] = 100
print(dict1)
print(dict2)
# {'0': 100, 'b': 20}
{'0': 100, 'b': 20}
```

5. What is cloning? Difference betn dict2 = dict1 & dict2 = dict1.copy()?

→ Cloning means creating a separate copy of a dictionary

ex:- cloning :-

```
dict1 = {"a" : 10}
dict2 = dict1.copy()
dict2["a"] = 100
print(dict1)  # {'a' : 10}
print(dict2)    {'a' : 100}
```

Difference:-

dict2 = dict1

1) Aliasing
2) same object
3) changes affect both
4) is → True

dict2 = dict1.copy()

1) cloning / copying.
2) New dictionary object
3) change to top-level entries don't affect original
4) is → false.

5. How does the in operator work with a dictionary?

→ The in operator searches keys, not values.

ex:-
```
Student = { "name" : "pratik",
            "age" : 22
          }
print("name" in student)  # True
print("pratik" in student) # false.
```

7. Difference bet" is and == When comparing dictionaries?

→ 1) == check whether the content are equal.
2) is check whether two variable refer to the same object in memory.

ex:-
```
dict1 = {"a" : 10}
dict2 = {"a" : 10}
```

```
print (dict1 == dict2)   # True
print (dict1 is dict2)   # false.
```

e.x :-

```
dict2 = dict1
print (dict1 == dict2)   # True
print (dict1 is dict2)   # True.
```

8. How can you check whether a dictionary is empty using the "not" operator?

→ 1)
```
d={}
if not d:
    print ("dictionary is empty")
# dictionary is empty.
```

2)
```
d= {"a" : 10}
if not d:
    print ("empty")
else:
    print ("Not empty")
# Not empty.
```

9. What is the use of the update() method?

→ The update() method is used to add new key-value pair or modify existing values.

```
ex:- student = {
        "name": 'pratik",
        "age" : 22
     }
student.update ({"city" : "pune"})
print (student)
```

- If the key already exists, it's value is updated/replaced.

ex:- student.update({"age":23})
print(student)

10. Difference bet^n pop(key), popitem() & clear()?

1) pop(key): Removes a specific key.

```
d = {"a":10, "b":20, "c":30}
d.pop("b")
print(d)
# {"a":10, "c":30}
```

2) popitem(): Remove & return the last inserted key value pair.

```
ex:- d={"a":10, "b":20, "c":30}
     d.popitem()
     print(d)
# {'a':10, 'b':20}
```

3) clear(): Removes all elements from the dictionary.

```
ex:- d={"a":10, "b":20, "c":30}
     d.clear()
     print(d)
     # {}
```

* Session 20: Dictionary (3):

1. What does the len() function return when used on a dictionary?

→ len() return the number of key-value pair in the dictionary.

Date
Page

```
student={"name":'prathik","age":22,"city":"pune"}
print(len(student))
#3.
```

2. When you use min() or max() on a dictionary, does it work on keys or value?

→ min() and max() work on the keys, not the value.

```
ex:- d={30:'A',10:'B',20:'C'}
     print(min(d))   #10
     print(max(d))   #30
```

②
```
d={"banana":10,"apple":20,"mango":30}
print(min(d))  # apple
print(max(d)) # mango.
```

3. What does sum() return when used on a dictionary? What happens if the keys are string?

→ sum() add the dictionary keys.

```
d={10:'A',20:'B',30:'C'}; print(sum(d))
print(d)
#60
```

If the keys are strings:

```
d={"a":10,"b":20}
print(sum(d))  # TypeError.
```

4. What is the difference bet^n any() & all() when used on a dictionary?

→ Both function checks the keys of dictionary.

- any() → return True if at least one key is truthy.
- all() → return True only if all keys are truthy.

```
ex:- d={0:"A",1:"B"}
     print(any(d))  # True
     print(all(d))  # false.
```

Date
Page

5. In a dictionary like {0: "a", 0: "b", False: "d"}, Why does any() return False?

→ any() checks keys and 0 is falsy key.

6. What does all() return if even one key of the dictionary is 0 or false?

→ all() return "False" if even one keys is 0 or false.

ex:- d = {1: "A", 2: "B", 0: "C"}
print (all (d))
false.

7. What does sorted() return when used on a dictionary? Does it change the original dictionary?

→ sorted() return a list of sorted keys.

d = {30: 'C', 10: 'A', 20: 'B'}
result = sorted(d)
print (result) [10, 20, 30]
print (d) {30: 'C', 10: 'A', 20: 'B'}
Note: sorted() does not change the original dictionary.

8. What is difference bet? sorted() & reverse() in dictionary?

sorted()	reverse()
1) Sorts dictionary key	1) Reverse dictionary iteration order.
2) Return a list	2) Return a reverse iterator
3) order is based on key value	3) order is based on original order.
4) sorted (d)	4) reverse (d)

e.x:- d={30:"C", 10:"A", 20:"B"}
print(sorted(d)) # [10,20,30]
~~print(sorted~~ print(list(reversed(d)) # [20,10,30]

9. Why do we use list() with reversed() to print the result?

→ We use list() to convert the iterator returned by reversed() into a list so that it's elements can be displayed and reused easily.

10. How can you print the key-value pairs of a dictionary in sorted order of keys?

→ 1) d={30:"C", 10:"A", 20:"B"}
for key in sorted(d):
print(key, d[key])

10 A
20 B
30 C

2) for key, value in sorted(d.items()):
print(key, value)

10 A
20 B
30 C

* Session 20: list Comprehension:

1. What is a comprehension in python? Why do we use it.

→ A comprehension is short and simple way to create a new collection from an iterable using a single expression.

- It is used to:
1) Write code in fewer lines.
2) Make simple transformation easier to read.
3) filter element easily.

e.x:-
```
square = [x ** x for x in range (1,6)]
print (square)
# [1,4,9,16,25]
```

2. Which container data type can be created using comprehension?

→ list, set, dictionary.

e.x:-
```
[x * 2 for x in range(5)] # List

{ x * 2 for x in range(5)] # set

{ x: x * x for x in range (5)] # dictionary.
```

3. In there a tuple comprehension in python? What do you get if you write a comprehension inside round bracket ()?

→ No, python does not have tuple comprehension.
A comprehension-like expression inside () create a generate expression.
- If you write
```
result = (x * 2 for x in range(5))
print (result)
```
- You get a generator object, not a tuple.

4. What is a list comprehension ? Explain it's syntax.

→ A list comprehension is concise way to create a list from an iterable.

- Syntax:-

[expression for item in iterable if condition]

- e.x:-

```
squares=[x * x for x in range (1,6)]
print (squares)
# [1, 4, 9, 16, 25]
```

5. In the syntax:- [exp for item in iterable if condition], explain the role of each part.

→ exp :- What value should be stored.
item :- receives each value.
Iterable : source of values.
condition : filters values.

6. Is the if condition compulsory in a list comprehension

→ No, The if condition is optional. It is used when we want to filter element.

e.x:-
```
num= [x *2 for x in range(1,6) if x>2]
print (num)
[6, 8, 10]
```

* What is the difference bern creating a list using a for loop with append() & using a list comprehension?

→ Both produce same result:-

1) using for loop:

```
square=[]
for x in range (1,6):
    square.append (x * x)
print (square).
```

2) Using list comprehension:

```
square = [x * x for x in range(1,6)]
print(square)
# [1, 4, 9, 16, 25]
```

3) Difference.

for loop + append()	list comprehension
1) more line	1) fewer line.
2) easy for complex logic	2) Best for simple logic
3) use append()	3) No append() needed

8- can a list comprehension use a function or method in it's expression such as upper()?

→ Yes,
A function or method can be used in the expression.

```
- ex:- names=['pratik", "rahul", "amit"]
       result=[name.upper() for name in names
       print(result)
```

9. What type of iterables can be used in a list comprehension?

→ A list comprehension can Work with any iterable.
common example:
1) list 2) tuple 3) string 4) set
4) Dictionary 5) range()

e.x:-
```
d= {"a":1, "b":2}
result = [key for key in d]
print (result)   # ['a', 'b']
```

10. What are the advantage of using list comprehension over a normal for loop?

→ 1) shorter code
2) Readable - Simple transformation can be expressed clearly.
3) filtering easy :- condition can be included directly
4) No need for append()
5) useful for data transformation.

* Session 21: Set & dictionary Comprehension.

1) What is set comprehension? How is it different from a list comprehension?

→ Set comprehension is a short way to create a set using a single expression.

-ex:-
```
number=[1,2,2,3,4]
S={x for x in numbers}
print(S)
# {1,2,3,4}
```

*Difference:

Set comprehension	List comprehension.
1) create a set	1) create a list
2) Uses {}	2] Uses []
3) Removes duplicates	3) keeps duplicates.

2. Explain the syntax of a set comprehension.

→ The basic syntax is:

{expression for item in iterable if condition}

-e.x:-

```
num = [1,2,3,4,5]
even = {x for x in num if x % 2 == 0}
print(even)
# {2,4}
```

3. What happens to duplicate values when you create a set using set comprehension?

→ set comprehension automatically removes duplicate values because set contain only unique element.

e.x:-
```
num = [1,2,2,3,3,3]
result = {x for x in num}
print(result)
# {1,2,3}
```

4. Why can't you rely on the order of element in the output of a set comprehension?

→ We should not depend on set ordering because. Sets do not guarantee insection order for iteration

5. How do you create a set without using comprehension? Which method is used to add element?

→ We can create an empty set using:

S=set() and use range function to iterate the element.

and we use the add() method to add a single element.

Date
Page

ex:- new_set=set()
for i in range(1,11):
if i<=5:
new_set.add(i**2)
print(new_set).

——— || Dictionary comprehension || ———

5. What is a dictionary comprehension? explain it's syntax?
→ Dictionary comprehension is a short way to create dictionary using an expression.

-
{key_exp: value_exp for item in iterable if condition}

- ex:- num=[1,2,3,4]
result={x: x*x for x in num}
print(result)
# {1:1, 2:4, 3:9, 4:16}

7. In a dictionary comprehension, what are the key expression & the value expression?
→ The key expression creates the dictionary key, & the value expression create the corresponding value.

num=[1,2,3]
result={x: x*10 for x in num}
here
x → key expression
x*10 → Value expression.
& In dictionary comprehension, the expression before : creates the key & the expression after : create the value.

8. Both set comprehension & dictionary comprehensi use curly braces {}. How does python know whi one you are creating?

→ Python looks at the structure inside the curly braces.

1) Set comprehension:

{ x for x in num }

There is only one expression, so python create set.

2) Dictionary comprehension:

{ x: x*2 for x in num }

There is a key-value pair separated by :, so python creates a dictionary.

9. Is the if condition compulsory in set & dictionary comprehensions? Give an example where condition is used?

→ No, The if condition optional.

without condition:

squares = {x*x for x in range(1,5)}

With condition.

even_square = {x*x for x in range(1,6) if x%2==0}

print(even_square). # {4,16}

In dictionary:

result = {x: x*x for x in range(1,6) if x%2==0}

print(result)

{2:4, 4:16}

10) What happens in a dictionary comprehension if the same key is produced more than once.

→ Dictionary keys must be unique. If the same key occurs again in a comprehension, the latest value overwrites the previous value.

e.x:-

```
num = [1, 2, 3]
result = {x % 2 : x for x in num}
print (result)
```

1 % 2 → 1 : 1
2 % 2 → 0 : 2
3 % 2 → 1 : 3
{1 : 3, 0 : 2}

* session 22 : python function:

1. What is a function in python? Why do we use function?

→ A function is block of reusable code that perform a specific task.

* We use function to:
- Avoid writing the same code repeatedly.
- Make code reusable.
- Make program easier to understand.
- Reduce code duplication.

eg:-
```
def greet():
    print ("Hello, python")
greet().
```

2. What are the advantage of using function? Explain code reusability & modularity.

→ ⇒ code reusability: Write code onces & use it multiple times.

2> Modularity - Divide a large program into smaller parts.
3> Readability: Makes code easier to understand
4> Maintainability: easy to modify or update.
5> Debugging :- Error can be isolated to particular function.

* code Reusability example:

```
def add(a,b):
    return a+b
print(add(10,20))
print(add(50,60))
```

The add() function is written once but used multiple time.

* Modularity:

Instead of writing one large program, we can divide it into function such as:

```
login()
calculate_salary()
generate_report()
send_email()
```

Q.3> What is difference betn built-in function & user defined function?

→

Built-in function	user-defined fun.
1> Already provided by python	1> create by the programmer
2> No need to define them	2> We define them using def.
3> Used directly.	3> we call them after defining.
4> e.x: len(), print()	4> Calculate()

Q4) How do you define & call a function in Python? Explain the Syntax.

→ We define a function using the def keyword.

Syntax:

```
def function_name(parameters):
    statement.
```

* Calling a function:

```
function_name(argument)
```

e.g.:

```
def greet(name):
    print("Hello", name)
greet("pratik").
```

Q.5) What is the naming convention for function names in python?

→ python generally follows snake_case for function names.

- Rules:
  - Use lowercase letter.
  - Use underscoces to separate words.
  - choosing meaningful names.
  - Don't use python keywords.
- ex:- Calculate_Salary()
  get_student_details()
  Send_email().

6. What is docstring? Why is it used in a function?

→ - A docstring is a string written inside a function to descibe what the function does.
- It is usually written using triple quotes.

Date
Page

ex:- def add(a,b):
    """This function return the Sum of two number."""
    return a+b

* We can view the docstring using.
  print(add.__doc__)

7. What happens if two functions are defined with the same name in a program?

→ If two function have same name in the same scope, the latest definition replaces/rebinds the previous one.

ex:- def greet():
        print("Hello")
    def greet():
        print("Welcome")
    greet()

Welcome.

** - python does not support traditional function overloading by defining multiple function with the same name & different parameter.

8. What is the difference bet^n parameter & arguments?

→

parameter	arguments.
1) Variable defined in a function definition	1) Actual values passed during function call.
2) Receive values	2) provide value.
3) Exist in the function defination	3) Exist in the function call.

ex:-

```
def add (a,b):
    return a+b
add (10,20).
```

here,
a & b → parameters.
10 & 20 → arguments.

9. What is the use of the return keyword? What does a function return if there is no return statement?

→ The return keyword is used to send result / value back from a function to a caller.

- ex:-
```
def add(a,b):
    return a+b
result = add (10,20)
print (result).
```

30.

- If there is no return statement, the function automatically return None.

ex:-
```
def greet():
    print ("Hello")
result = greet()  # Hello
print (result)  # None.
```

10. What is the difference bern position argument, keyword arguments & default argument?

→ A. position argument: Value are passed accoding to their position lorder.

```
def student (name, age):
    print (name, age)
student ("pratik", 23).
```

B. Keyword Argument: Values are passed using the parameter name.

```
ex:- def student(name, age):
        print(name, age)
     student(age=23, name="pratik")
```

C. Default Argument:

- A parameter is given a default value in the function definition. If the caller doesn't provide the argument, the default value is used.

```
ex:-
def greet(name="pratik")
    print("Hello", name)
greet()  # pratik
greet("user") # Hello user
```

* Session 23: Variable length Argument

1) What are variable length positional argument? When do we use them?

→ Variable length argument allows a function to accept any number of positional argument.

- We use them when we don't known in advance how many value the user will pass.
- We use the * symbol.

```
e.g:- def add(*args):
          return sum(args)
      print(add(20,30)) # 50
      print(add(10,20,30,40)) # 100.
```

2) What is *args? In which data type does python stored the value received in *args?

→ *args is used to accept a variable number of positional argument.
- python stores all the value received by *args in the tuple.
-ex:- def display(*args):
        print(args)
        print(type(args))
      display(10,20,30)
# (10,20,30)
<class 'tuple'>

3. What is the difference bern a normal positional argumen & *args?

→

Normal positional argument	*args.
1) Usually accept a fixed number of values.	1) Accept variable numb er of value.
2) each parameter repres ent one value.	2) one parameter can collect multiple values.
3) example: def add(a,b)	3) example: def add(*args).

```
* Normal:
def add(a,b):
    return a+b
add(10,20)
```

```
* Variable
def add(*args):
    return sum(args)
add(10,20,30,40).
```

4. Is the compulsory to use the name args & *args?
→ No, the name args is not compulsory.
-But *args is the standard & commonly used convention.

ex:- def add (*num):
print (num)
add (10,20,30).

5. Can a function with *args be called with zero arguments?

→ Yes,
A function using *args can be called with zero arguments.
- In that case, args will contain an empty tuple.
- ex:- def display(*args):
print(args)
display()
# ()

6. What are variable length keyword argument? When do we use them?

→ Variable length keyword argument allow a function to accept any number of keyword argument.
- They are represented using **
- we use them when we don't knows in advance how many key-value pair will be passed.
ex:- def student (**kwargs):
print (details)
student (name="pratik", age=22, course="MCA"

7. What is **kwargs? In which data type does python store the values received in **kwargs?

→ ** kwargs is used to accept a variable number of keyword argument.
- python stores them in dictionary (dict).

8. What is the difference betn *args & **kwargs?
→

*args	**kwargs.
1) Accept variable positional argument	1) Accept variable keyword argument.
2) use one *	2) use two **
3) stores values in a tuple.	3) stores values in a dictionary.
4) Doesn't require parameter name	4) Requires key/parameter name.

ex:- def demo(*args, **kwargs):
print(args) # (10, 20)
print(kwargs) # {'name': 'pratik', 'age': 23}
demo(10, 20, name="pratik", age=23)

9. How can you loop through the values received in **kwargs?
→ Since **kwargs is stored as a dictionary, we can use a for loop.

ex:- def student(**kwargs)
for key, value in kwargs.items():
print(key, ":", value)
Student(name="pratik", age=22, course="MCA")
# name: pratik
age=22
course=MCA.

Date
Page

10. can you use built-in functions like len() & sum() on *args?

→ Yes,
Because *args is stored as a tuple we can use function such as len() & sum() on it.

e.x:

```
def calculate(*numbers):
    print("values:", numbers)
    print("count:", len(numbers))  # 4
    print("sum:", sum(numbers))  # 100.
calculate(10,20,30,40)
```

* Session 23: Lambda & Map.

1. What is functional programming in python? Name the functions covered under it.

→ functional programming is programming style where we use functions as the main building blocks of a program.

- python support function programming features such as:
  - lambda()
  - map()
  - filter()
  - reduce()

e.x:-

```
num=[1,2,3,4]
result=list(map(lambda x: x*2, numbers))
print(result)
# [2,4,6,8]
```

- functional programming is a programming style in which use function perform operation on Data.

2. What is a lambda function? Why is it called an anonymous function?

→ A lambda function is small, single-expression function created using the lambda keyword.
- It is called an anonymous function because it normally does not have a function name like a function created with def
- e.x':- square = lambda x: x * x
print (square(5))
25.

3. Explain the syntax of a lambda function:

→ Syntax:
lambda arguments : expression.

e.x':-
add = lambda a, b: a+b
print (add(10,20))
#30

4. main characterstics of a lambda function.

→ 1) It is created using lambda keyword
2) It is usually an anonymous function.
3) It can take multiple arguments.
4) It contains only one expression.

5. difference normal function & lambda function.

normal function	lambda function.
1) create using def	1) create using lambda.
2) usually has a name	2) usually anonymous.
3) contain the multiple statement	3) contain one expression.

4) Can have docstring	4) Not normally used for docstring.
5) Can use return	5) No explicit return.

6. Do you need write the return keyword in a lambda function?

→ No,
The result of the lambda expression is automatically returned.

```
ex: add = lambda a,b: a+b
    print(add(10,20))
    #30.
```

7. Can a lambda function take more than one argument?

→ Yes.

```
multiply=lambda a,b: a*b
print(multiply(5,4))
#20
```

Map()

8. What is the map() function? explain it's syntax.

→ The map() function applies a given function to every element of an iterable, such as a list or tuple.

- syntax:

```
map(function, iterable).
```

- ex:-

```
num = [1,2,3,4]
result = map(lambd x: x*2, num)
print(list(result))
# [2,4,6,8]
```

9) What does map() return? How do you convert the result into a list, set or tuple.

→ In python 3, map() return a map object, which is an iterator.

e.x :- num = [1,2,3]
result = map (lambda x: x*2, num)
print (result)

e.x convert to list.

result = list (map (lambda x: x*2, num))
print (result) # [2,4,6]

e.x: convert to tuple:
result = tuple (map (lambda €x: x*2, num))
print (result) # (2,4,6)

10. Can We use both a normal function & a lambda functio With map()?

→ Yes.
map() can accept both a normal function & a lambda function.

A. using normal function With map()
def square (x):
    return x*x
number = [1,2,3,4]
result = map (square, numbers)
print (list (result))
[1,4,9,16]

Date
Page

8] using lambda() with map()

```
num = [1,2,3,4]
result = map (lambda x: x*x, num)
print(list(result)
# [1,4,9,16].
```

* Session 24: filter and reduce.

1) What is the filter() function in python? explain it's Syntax.

→ • filter() is built-in python function.
• It is used select/filter element from an iterable based on a condition.
• It keeps only those element for which the condition is True.

-Syntax:-

filter(function, iterable)

e.x:- number=[1,2,3,4,5]

```
def is_even(n):
    return n%2 == 0
result = filter(is_even, numbers)
print(list(result))
```

* [2,4].

2) What should the function passed to filter() return?

→ -The function passed filter() should return Boolean value.
• True → element is kept
• false → element is removed

ex:-

```
def is_even(n):
    return n%2==0
```

For [1,2,3,4]

1%2==0 False

2%2==0 True

3%2==0 False

4%2==0 True.

3. What does filter() return? How do you convert the result into a list?

→ 1) filter() rerurn a filter object, which is an iterator.

2) To convert it into a list, use list().

ex:-

```
number=[1,2,3,4,5]
result=filter(lambda x: x>3,number)
print(result)
print(list(result)).
```

<filter object....>

[4,5]

4. What is the difference bern map() and filter()?

→

map()	filter()
1) Used to transform elements.	1) used to select elements.
2) Applies a function to every element.	2) check a condition for every element.
3) can change the Values	3) Usually keeps the original values.
4) Return a map object	4) Return a filter object
5) ex:- square number	5) ex:- select even number.

*map() example:-

```
number=[1,2,3]
result= map(lambda x: x*2, number)
print(list(result))
# [2,4,6]
```

* filter():

```
number=[1,2,3,4]
result= filter(lambda x: x%2==0, number)
print(list(result))
# [2,4]
```

Q.5 can we use both a normal function & lambda function with filter()?

→ Yes, we can use both.

* using normal function:

```
num=[1,2,3,4,5]
def is_even(n):
    return n%2==0
result= filter(is_even, num)
print(list(result))  # [2,4]
```

* using lambda function:

```
num=[1,2,3,4,5]
result= filter(lambda n: n%2==0, num)
print(list(result))
# [2,4]
```

——— || Reduce || ———

Q.6) What is the reduce() function ? Explain it's syntax?

→ 1) reduce() is used to reduce multiple element into a single value.

2) It repeatedly applies a function to the element.

- Syntax : reduce(function, iteroble).

e.x:- from functools import reduce.

```
num=[1,2,3,4]
result=reduce(lambda x,y:x+y,num)
print(result)
#10.
```

Q.7) from which ~~module~~ module do we import reduce()?

→ reduce() is available in the functools module.

* Import statement:

~~fun~~ from functool import reduce.

Q.8] explain step by step how reduce() works on [1,2,3,4,5] with addition.

→

```
from functool import reduce
num=[1,2,3,4,5]
result=reduce(lambda x,y:x+y,num)
print(result)
```

* step: [1,2,3,4,5]

1+2=3

3+3=6

6+4→10

10+5→15

Q.9] How many argument must the function passed to reduce() take? why?

→ The function passed to reduce() must take two argument.

e.x:-
lambda x,y: x+y

Q 10. What is the difference bet^n the output of map(), filter() and reduce?

→

	function	purpose	output
1)	map()	Transforms every element	map object.
2)	filter()	selects element based on condition	filter object
3)	reduce()	combines element into one value	single value.

* Session 24: Exception Handling.

Q.1) What are the two type of error we get while working in python?

→ There are mainly two type of error:
1) syntax error
2) Exception (Runtime error)

e.x:- print("Hello World") → Syntax error.
a = 10/0 → ZeroDivisionError Exception

Q.2) What is the difference bet^n a syntax error and an exception (runtime error)?

→

Syntax error	Exception.
1) Occurs when python syntax is incorrect.	1) Occurs while the program is running
2) program cannot start properly	2) program starts but encounters a problem.
3) usually detected before execution	3) detected during execution.
4) e.x': missing ')'	4) example: 10/0.
5) cannot normally be handle with try-except at the the point invalid syntax	5) can be handled using try-except.

e.x:- syntax error.

```
if 10>5
    print("Yes")
```

e.x': exception.

```
print(10/0).
```

Q.3) What happen to the rest of the program when an exception occurs and it is not handle?

→ If an exception occurs and it is not handled:

- python stops the execution of the current program.
- The statements after the exception are not executed.
- python displays an error message | traceback.

e.x:-

```
print('start')
a=10/0
print("end")
```

start

ZeroDivisionError: division by zero.

Q.4) What is exception Handling? Why do We need it?

→ Exception handling is mechanism used to handle

runtime error without suddenly terminating the program.

- python provide
  try
  except
  else
  finally.

* Why do we need it?

1) prevents sudden program termination.
2) Handles runtime error safely.
3) Allows the program to continue when possible.
4) provide meaningful error message.
5) helps handle unexpected situation.

example:

```
try:
    a=1010
except zeroDivisionError:
    print("cannot divide by zero")
```

Q.5) Explain the syntax of try and except. What type of code should be written inside the try block?

→ * Syntax:

```
try:
    # code that may cause an exception
Catch:
    # code to handle the exception.
```

e.x:-

```
try:
    num=int(input("enter number: "))
    result=10/num
    print(result).
```

```
except:
    print("An error occurred")
```

- The try block should contain only the statement that may cause an exception.

Q6. What is the use of "as err" in an except block?

→ "as err" is used to store the exception object in a variable.

- It allows us to see or print the actual error message.

- example:

```
try:
    result=10/0
except Exception as err:
    print(err)
# division by zero.
```

Q.7) When is zeroDivisionError raised.

→ zeroDivisionError occur when we try to divide a number by zero.

ex: 1 =

```
a=10
b=0
print(a/b)
# zeroDivisionError: division by zero.
```

Q.8) When is OverflowError raised? why does it occur with float value & not with int value?

→ OverflowError occurs when a numeric calculation produce a result that is too large for floating point number to represent.

ex:-
```
import math
print(math.exp(1000))
# OverflowError.
```

⇒ OverflowError occurs when a numerical operation produces a value too large for the available floating-point representation. python integers can grow to very large sizes, while floating-point numbers have limited range.

Q.9. When is TypeError raised? Give two example?

→ TypeError occurs when operation or function is used with an inappropriate or incompatible data type.

ex:- adding string & integer.

```
a="10"
b=20
print(a+b) # TypeError
```

② Calling an integer like a function:

```
num=10
num()
# TypeError.
```

Because an integer is not callable.

10. If an exception is handled using try and except, will the statement written after the try except block be executed?

→ Yes.

ex:-
```
print("Before")
try:
    result=10/0
except ZeroDivisionError:
    print("exception handled")
print("After")
```
Before
Exception handled
After.

* Session 25: Exception handling-2

Q.1) What is a ValueError raised? How is it different from a TypeError?

→ * ValueError:
- Raised when a function gets the correct data type but an invalid value.
- example:
```
num=int("ten") # ValueError.
```

* TypeError:
- Raised when an operation is performed on the wrong data type.
```
a= 10+"20" #TypeError.
```

Q.2) When is an AttributeError raised? Given example.
- Raised when we try to use a method or attribute that does not exist for an object.

ex:-
```
list1=[1,2,3,4]
list1.upper()
```
upper() method belongs to string not list.

Q.3) When is an IndexError raised? Which data type can raise it?

→ • Raised when we access an index that does not exist.

```
ex:- list = [10,20,30]
     print(list[5])  # IndexError.
```

* list, tuple, string data type raise IndexError.

Q.4) When is a KeyError raised? How is it different from an IndexError?

→ • Raised when we try to access a key that does not exist in a dictionary.

```
- ex:- student = {"name": "pratik"
                  "age": 23
                 }
      print(student["Roll_no"])  # KeyError.
```

* Difference:

KeyError	IndexError
1) Occurs in dictionaries	1) occur in sequences
2) Uses key	2) Uses index
3) dict["x"]	3) list[10]

Q.5) What is difference bet^n ModuleNotFoundError and ImportError?

→ * ModuleNotFoundError:

• Raised when python cannot find the module.

ex:- import Langchain # ModuleNotFoundError

* ImportError:

• Raised when the module exists, but a specific object cannot be imported.

e.x:-

```
from pandas import D   # ImportError.
```

Q.6) If you get a ModuleNotFoundError for an external library, what is the most common reason & how do you fix it?

→ * common reason.

• The library is not installed

```
import pandas.
```

If pandas is not installed:

```
pip install pandas.
```

* other reason:

• Wrong python environment.

• Typing mistake in module name.

Q.7) What is the difference bet^n handling a specific exception & generic exception?

→

Specific Exception	Generic Exception.
1) Handle one error	1) Handle all error
2) Easier debugging	2) Less precise.
e.x:-	3) e.x:

```
try:
    num = int(input())
except ValueError:
    print("Invalid Number").
```

```
try:
    num = int(input())
except Exception:
    print('some error
    occurred').
```

8. What does type(err).__name__ return? Why is it useful?

→ e.x :-

```
try:
    1/0
except Exception as err:
    print(type(err).__name__)
# ZeroDivisionError.
```

* It return: The name of the exception class.
* Useful because:
- helps identify the exact error.
- Useful for debugging.
- Can display error names dynamically.

9. If a try block has three statement and the second one raised an exception, will the third statement run?

→ No,

e.x :-

```
try:
    print("statement 1")
    x=1/0
    print("statement 2")
except ZeroDivisionError:
    print("error occur")
# statement 1.
  error occue.
```

10. When is KeyboardInterrupt raised ? Name a few other exception?

→ * KeyboardInterrupt:
- Raised when the user interrupts the program manually.
- Usually by pressing..
Ctrl + C

-example:-
While True:
    pass
press ctrl + c → keyboard Interrupt.
* other Exception:
1) file handling → fileNotfound Error.
2) file Handling → permission Error.
3) file Handling → IsADirectoryError
4) Recuesion → RecursionError
5) Unit Testing → AsseetionError.

* Session 26: file Input/output:

1. Explain how file I/O works in python. What role do open(), the memory buffer and close() play?
→ 1) file I/o stands for file input/output. It is used to read data from files and write data into files.
* The main step are:
1) open() - opens a file and return a file object.
2) Memory buffer: Temporarily holds data while reading or writing.
3) Read/Write operation: Reads data from the file or write data to it.
4) close(): closes the file and releases associated resources.
e.x:- f = open ("demo.txt", "w")
f.write ("Hello python")
f.close.

2. What is the difference bet "r", "w" and "a" modes in open()?

→ These modes specify how we want to work with a file.

"r" → read → Reads existing file data.
"w" → Write → Create a file or overwrites existing content.
"a" → Append → Adds data at the end of the data.

e.x:-

```
open("demo.txt", "r")
open("demo.txt", "w")
open("demo.txt", "a").
```

Q3) What happen if you open a file in "w" mode that already contain data? What happens if the file does not exist?

→ 1) If the file already exists "w" mode removes the existing content before writing new data.
2) If the file does not exist, python creates a new file, provided the directory exists and permission allow it.

e.x:-

```
with open("demo.txt","w") as f:
    f.write("New Data")
```

Q4) What is the difference bet^n read(), read(n) & readline()?

→ read() → Reads the entire remaining file content
read(n) → Reads up to n characters in text mode.
readline() → Reads one line at a time, including the newline character if present.

demo.txt:
Hello Python
Welcome.

Date .....
Page .....

```
With open("demo.txt", "r") as f:
    print(f.read())
# Hello Python
  Welcome.
```

using read(5) return:
Hello

using readline() return the first line:
Hello Python.

Q.5) Why is it risky to work with files without using the with keyword?

→ 1) We may forget to close the file.
2) System resource may remain occupied longer than necessary
3) Exception may prevent close() from running.
4) Buffered output may not be flushed when expected if the file is not properly closed.

e.x:-
```
f = open("demo.txt", "w")
    f.write("Hello")
    f.close()
```
If an exception occurs before close(), the file might remain open.

Q.6) What are the advantage of using the With keyword for file handling?

→ 1) Automatically closes
2) work even if an exception occur Inside the block
3) Reduces the chance of resource leaks.
4) Makes code shorter & cleaner.
5) helps manage file resource properly.

e.x :- With open ("demo.txt", "w") as f:
f.write ("Hello python")

Q.7) How do you check whether a file exists before opening it?

→ -We can use os.path.exists() or pathlib.path.exists().

e.x :- import os
if os.path.exists ("demo.txt"):
print ("file exists")
else:
print ("file does not exist").

② if os.path.isfile ("demo.txt"):
print ("It is file")

- exists() checks whether a path exists
isfile() checks whether a is a file exist.

Q.8) Difference os.mkdir() & osmakedirs()?

→

os.mkdir()	os.makedirs()
1) create one directory	1) create nested directory.
2) parent directory must already exist	2) can create missing parent directory.
e.x import os os.mkdir ("folder")	e.x :- os.makedirs ("Parent/child/data")

Q.9) What is the difference bet n os.path.isfile() & os.path.isdir()?

→ 1) os.path.isfile (path) : Return True if the path is a file.

2) os.path.isdir (path) : Return True if the path is directory.

ex
```
import os
print(os.path.isfile("demo.txt"))
print(os.path.isdir("folder"))
```

Q.10) What is the use of following ~~do~~ function?

→
1) os.getcwd :- Gets the current working directory
2) os.chdir :- changes the current working directory
3) os.listdir :- lists files & directories inside a specified directory.

* Session: 27 :- class and object

Q.1) What is the difference bet^n the procedual approach & the OOP approach?

→ *produral approach :- A program is divided into function and the program executes step by step.
* OOP (object-oriented programming) : A program is organized using classes & object that combine data & method.

procedural Approach	OOP Approach
1) focus on function	1) focus on objects.
2) Data & function are generally separate	2) Data & method are grouped in classes
3) Suitable for small program.	3) Useful for large & Complex program.
4) e.x : C	4) e.x : python, Java.

Q.2) What does "everything in python is an object" mean?

→ It means that values in python are object. They have a type & support particular operation

And methods?

e.x :-

```
x = 10
print(x)          # 10
print(type(x)) # <class 'int'>
print(x.bit_length()) # 4
```

⇒ In python, number, string, list and function are object with their own type & behavior.

Q.3 Name the main principle ob OOP?

→ 1) Encapsulation: combining data and method in a class & controlling access to them.
2) Inheritance: one class can acquire feature of another class
3) polymorphism: The same method or interface can behave differently in different situation.
4) Abstraction :- Hiding implementation details & showing essential feature.

Q.4) What is class? Why is it called blueprint?

→ 1) A class is template used to create objects. It defines the attribute and methods that it's object can have.

e.x :- class student:
name="pratik".

here student is class.

2) It is called a blueprint because it defines how object can be creates, just as a house blueprint describes how a house will be built.

Q.5) What is an object? Why is it called an instance of class?

→ 1) A Object is a specific entity create from a class.

ex:- class student:
    name = "pratik"
    s1 = student()
    print(s1.name)
    # pratik

2) It is called an instance because it is a particular object created from that class.

** 3) An object is an instance of a class that can have it's own data and use the methods defined by the class.

Q.6) What are two components of a class? explain attribute & methods.

→ The two main components are attribute and methods.
- Attribute: Variables that store data or properties.
- Methods: function defined inside a class that describe it's behavior.

ex:- class student:
    def __init__(self):
        self.name = "pratik" # Attribute.
    def study(self): # Method
        print("student is studying").
s1 = student()
print(s1.name)
s1.study().

Q.7) What is the __init__() method? When is it called?

→ 1) The __init__() method initializes an object attribute. python calls it automatically when

an object is created.

** → __init__() is an initializer method that runs automatically when object is created. It is commonly used to initialize instance attribute.

Q.8) What is self? Why do we write it as the first parameter in every method of class?

→ 1) self refers to the current object. It allows an instance method to access that object attribute & other methods.

e.x.-
```
class student:
    def show(self):
        print("Hello student")
s1 = student()
s1.show()
```

** → self refers to the current instance and help access it's attribute & methods.

Q.9. How do you create multiple object of the same class with different values?

→ Yes, We can create multiple object by calling the same class with different argument. each object maintains it's own instance attributes.

-e.x.-
```
class student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
s1 = student("pratik", 23)
s2 = student("om", 22)
print(s1.name, s1.age)
print(s2.name, s2.age)
```

10. How do you access an attribute & call a method ob an object.

→ We use dot notation (.)

- Syntax:

```
object_name.attribute_name
object_name.method_name()
```

- e.x:-

```
class student:
    def __init__(self, name):
        self.name=name
    def greet(self):
        print("Hello.", self.name)
s1 = student("Pratik")
print(s1.name) # Access Attribute.
s1.greet() # call method.

# Pratik
Hello Pratik
```

* Session 28: Class & object Part 2.

1. What is the difference bet^n a method & function?

→ *function: A block of reusable code that can be called independently.

*Method: A function defined inside a class & generally called using an object or class.

e.x:-

```
#function
def greet():
    print("Hello")
greet()
```

ex

```
# Method
class student:
    def greet(self):
        print("Hello student")
s1 = student()
s.greet().
```

2. Why are append() & remove() methods, while len() & sum() are function.

→ Because append() and remove() are called using a list object. While len() & sum() are built-in function called independently.

- ex:- numbers = [10,20,30]
  number.append(40) # Method
  print(len(number)) # function.
- Syntax:-
  - list.method() → Method
  - function(value) → function.

3. What is a constructor in python? Which method acts as the constructor?

→ A constructor is used during object creation to initialize an object's data.

- In python, --init--() is commonly called the constructor, although technically it is the initializer; --new--() is responsible for creating the object.
- ex:- class student:
  def --init--(self):
  print("student object initialized")
  s1 = student().

4. What is the different betn --init--() and a normal method?

→

--init--() method	Normal method.
1) Runs automatically when an object is initialized	1) Runs when we call it.
2) Usually initalizes object attribute	2) performs a particular task

3> commonly sets initial values

3> can calculate, display or update data.

-example.

```
class student:
    def __init__(self, name):
        self.name = name
    def display(self):
        print(self.name)
s1 = student("prahk")
s1.display().
```

5. Why do We Write configuration-related code inside the constructor?

→ We initialize the data or resource an object needs when it is created.

-for example: we can store database settings or initialize an API client inside __init__().

e.x'.-

```
class Database:
    def __init__(self, database_name):
        self.database_name = database_name
db = Database("College")
print(db.database_name)
# college.
```

6. What is self ? What does it refer to?

→ self refers to the current object of the class. It allow us to access the object's attributes & method.

-e.x'.-

```
class student:
    def __init__(self, name):
        self.name = name
s1 = student("prahk")
print(s1.name).
```

Pratik

⇒ self refers to the current instance of a class. It is a convention in python, not a reserved keyword.

7. How can you prove that self & the object refer to the same thing.

① we can compare self with the object using is

e.x:-
```
class student:
    def check(self):
        print(self is S1)

S1 = student()
S1.check()

# True.
```

e.x:-
```
class student:
    def check(self):
        print(id(self))

S1 = student()
print(id(S1))
```

8. In which three places do we use self inside a class.

→ We commonly use self in these three ways:
1> As the first parameter of an instance method
2> to access or create instance attribute.
3> to call another instance method of the same object.

- e.x:

```
class student:
    def __init__(self, name):      # 1. first parameter.
        self.name = name           # 2. Attribute.
    def display(self):
        self.show()                # 3. call another method.
    def show(self):
        print(self.name)
s1 = student("Pratik")
s1.display()
# Pratik.
```

9. How do you call one method from another method in the same class?

→ Use self.method_name().

-example:-

```
class calculator:
    def add(self):
        print(10+20)
    def result(self):
        self.add()
c1 = Calculator()
c1.result()
# 30.
```

10) What happens if you forget self as the first parameter of a method?

→ Python normally raise a TYPEError when you call the method through an object and the argument no longer match.

Ex:-
```
class student:
    def display():
        print("Hello")
s1=student()
s1.display()
```
TypeError: student.display() takes 0 positional arguments but 1 was given.

Session 30: Magic Methods.

1. What are magic method in python? Why are they called dunder method?

→ 1) Magic method are special methods in python that define how object behaves with built-in operation.

2) They are called dunder method because their name begin & end with double underscore (--)

-ex:- --init--(), --str--(), --add--()

2. When are magic method called? Do we need to call them manually?

→ python automatically calls magic method when we perform certain operations.

- Usually, We don't need to call them manually.

- ex:-
```
print(10+20)
```

3. How do magic method help us create custom data type?

→ Magic method allow us to define how objects of our classes behave with operations such as addition, subtraction, printing & comparison.

4. What is the use of --init--() ? When does it run?

→ The --init--() method initializes an object attribute. It runs automatically after a new object has been created.

- ex:-

```
class student:
    def --init-- (self, name):
        self.name = name
s1 = student("pratik")
print(s1.name)
# pratik
```

5. What is the use of --str--() ? What happens if it is absent?

→ The --str--() method defines the human-readable string representation of an object.

- class student:

```
    def --str-- (self):
        return "student object"
s1 = student()
print(s1).
# student object.
```

6. What must --str--() return?

→ the --str--() method must return a string (str).

7. What magic method is triggered by the + operator? What are self and other?

→ The --add--() method handles the addition operation for object of a class when the class support it.

- self: refer to the left-hand object.
- other: refer to the right-hand object.

e.x :-

```
class Number:
    def __init__(self, value):
        self.value = value
    def __add__(self, other):
        return Number(self.value + other.value
n1 = Number(10)
n2 = Number(20)
result = n1 + n2
print(result.value)
# 30
```

8. Which magic methods are called for -, *, and /

→

operator	Magic method.
+	__add__()
-	__sub__()
*	__mul__()
/	__truediv__()

9. Why does __add__() return a new object instead of printing the result?

→ - Returning a new object allows the result to be stored in a variable, reused and combined with other operations. printing only display the result & does not return it for further use.

- for e.x :- result = n1 + n2

10. What error occurss if we use + on object without __add__()?

→ e.x :-

```
class student:
    pass
```

s1 = student()
s2 = student()
print(s1+s2)
# TypeError: unsupport operand type(s) for + : 'student' + 'student'.

* Session 31 :- Linux session 1 :-

* Session 32 :- Instance, static & encapsulation :-

Q.1) What is the instance variable? Where is it defined & how do you access it?

→ -An instance variable is a variable that stores data specific to an individual object.
-It is usually defined inside the --init--() method using self.
-ex:-

```
class Employee:
    def --init-- (self, name):
        self.name = name
emp1 = Employee("pratik")
emp2 = Employee("Rahul").
print(emp1.name) # pratik
print(emp2.name) # Rahul.
```

Q.2) What is a static variable? Where is it defined & how do you access it?

→ -A static variable, also called a class variable, is shared by all object of a class.
-It is defined inside the class but outside instance method.

-e.x':-

```
class Employee:
    company = "TCS"
emp1 = Employee()
emp2 = Employee()
print(Employee.company)  # TCS
print(emp1.company) # TCS.
```

3. Difference bern instance and static Variable.

→

Instance Variable	Static Variable.
1) Belongs to an individual object.	1) Belongs to the class.
2) Defined using self	2) Defined directly inside the class
3) each object can have different value.	3) Shared by object by default.
4) e.x':- employee name	4) e.x': company name.

4. What instance method ? What is it's first parameter

→ An instance method is method that works with an object data. It's first parameter is self.

e.x':-

```
class student:
    def __init__(self,name):
        self.name = name
    def display(self):
        print(self.name)
s1 = student("prahik")
s1.display().
# prahik.
```

- An instance method works with instance variable.

5. What is a static method? How do you create & call it?
→ A static method is a method that does not required the instance (self) or class (cls) as it's first parameter. We create it using the @staticmethod decorator.

-e.x':-

```
class calculator:
    @staticmethod
    def add(a,b):
        return a+b
print(calculator.add(10,20))
# 30.
```

6. can a static method access instance variable or static variable?
→ • Instance variable: Not directly, because a static method does not receive self. It can access them if an object is explicitly passed to it.
• static variable: Yes, they can be accessed through the class name.

e.x':-

```
class Employee:
    company = "TCS"
    def __init__(self, name):
        self.name = name
    @staticmethod
    def show_company():
        print(Employee.company)
emp = Employee("prathk")
Employee.show_company()
# TCS.
```

⇒ A static method can access class variable using the class name, but it cannot directly access instance variable because it does not receive self.

7. When should we use a static method instead of an instance method?

→ We use a static method for utility function that do not need instance or class data. - we use an instance method when we need to access to modify an object data.

ex:

```
class Number:
    @staticmethod
    def is_even(n):
        return n%2 == 0
print(Number.is_even(10)) # True.
```

———— " encapsulation " ————

8. What is encapsulation? Why is it called "data hiding + controlled access"?

→ encapsulation:- encapsulation combines data & method into one class and provides controlled access to the data. It help protect data from invalid changes.

-ex:-

```
class BackAccount:
    def __init__(self, balance):
        self.__balance = balance
    def get_balance(self):
        return self.__balance
    def deposite(self, amount):
        if amount > 0:
```

```
    self.__balance += amount
account = BankAccount(1000)
account.deposite(500)
print(account.get_balance())
#1500
```

9 What problem occurs without encapsulation? How does the __prefix help?

→ Without encapsulation data can be changed incorrectly. The __prefix enables name mangling which help prevent accidental access to internal attribute. It does not provide absolute security.

- ex:-

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary
    def get_salary(self):
        return self.__salary
emp=Employee(30000)
print(emp.get_salary())
```

10. If we write obj.__salary=0 outside the class, does the actual salary change?

→ No, the actual salary does not change. Python uses name mangling for double-underscore attribute, so assigning obj.__salary outside the class normally creates a separate attribute. The original value remains unchanged.

- ex:-

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary
```

```
    def get_salary(self):
        return self.__salary
emp = Employee(500000)
emp.__salary = 0
print(emp.get_salary())
print(emp.__salary)
# 50000
# 0.
```

* Session 33: Getter & Setter.

1. What is encapsulation? Explain it using the formula: private attribute + getter method + setter method.

→ encapsulation is an oop concept that combines data and method inside a class and controls how the data is accessed or modified.

- formula:

private attribute + Getter method + setter method = Encapsulation.

- private attribute: stores data inside the class, such as __balance.
- Getter method: Reads and returns the value.
- Setter method: Updates the value with proper validation.

- example:

```
class Student:
    def __init__(self):
        self.__marks = 80
    def get_marks(self): # Getter
        return self.__marks
    def set_marks(self, marks): # Setter
        if 0 <= marks <= 100:
```

```
        self.__marks = marks
    else:
        print('Invalid marks")
s1 = Student()
print(s1.get_marks()) # 80
s1.set_marks(95)
print(s1.get_marks()) #95.
```

Q.2) What is getter method? why do we need it if a private variable cannot be accessed directly from outside the class?

→ - A getter method is a method used to read & return the value of private attribute.
- We use a getter because it provides a controlled way to access internal data without modifying it.
- ex:-

```
class Student:
    def __init__(self):
        self.__marks = 90
    def get_marks(self):
        return self.__marks
S1 = Student()
print(s1.get_marks()) # 90.
```

Q. What is a setter method? How is it different from directly changing a variable from outside the class?

→ A setter method is used to update the value of an attribute, usually after checking whether the new value is valid.

ex:-

```
class Student:
    def __init__(self):
        self.__marks = 80
    def set_marks(self, marks):
        if 0<= marks <=100:
            self.__marks = marks
        else:
            print("Invalid marks")
    def get_marks(self):
        return self.__marks
s1 = Student()
s1.set_marks(95)
print(s1.get_marks()) #95
s1.set_marks(150) # Invalid marks.
```

4. Why is it important to add validation inside a setter method?

→ Validation ensures that only acceptable values are stored in an object's attribute.

ex:-

```
class BankAccount:
    def __init__(self):
        self.__balance = 1000
    def set_balance(self, amount):
        if amount >= 0:
            self.__balance = amount
        else:
            print("Balance cannot be negative")
    def get_balance(self):
        return self.__balance.
```

```
account = BankAccount()
account.set_balance(-500)
print(account.get_balance())
# Balance cannot be negative
  1000
```

5. How does encapsulation protect the internal state of an object? Give a real-life example.

→ encapsulation protects an object's internal state by restricting direct access and allowing changes through methods that enforce rule.

* Real-life example:-

Bank Account
private attribute: --balance.
↓
Getter & setter Method
Read balance. validate change.
↓
protected Account data.
Reject invalid changes, such a negative balance when the rules prohibit it.

* Session 34: Inheritance:

1. What is inheritance in python? Why is it called an IS-A relationship?

→ - Inheritance is an oop concept in which a child class acquires the attribute & method of parent class. It is called IS-A relationship because the child is type of parent.

ex:- A Dog IS-A Animal

```
class Animal:
    def eat(self):
        print("Animal is eating")

class Dog(Animal):
    def bark(self):
        print("Dog is barking")

d = Dog()
d.eat()
d.bark()
```

2. What are the other names for the parent & child classes?

→ parent class
1) Base class
2) superclass

child class
1) Derived class
2) Subclass.

3. What are the main benefits of inheritance? What does DRY means?

→ * Benefits:
1) code reusability - reuse existing code.
2) Less code duplication.
3) easy maintenance
4) Suppose method overriding.
5) help create a clear class hierarchy.

* DRY principle: DRY stands for Don't Repeat Yourself.
-It means writing common code once & reusing it instead of repeating it.

4. Write the syntax to create a child class.

→ Syntax:-

```
class Parent:
    pass
class child(parent):
    pass.
```

ex:-

```
class Vehicle:
    def start(self):
        print("Vehicle started")
class Car(vehicle):
    pass
C=car()
C.start().
```

Q.5) Can child object call parent methods? can a parent object call child method?

→ • Yes, child object call parent methods
• No, a parent object cannot automatically call method defined only in the child class.

ex:-

```
class parent:
    def show(self):
        print("parent method")

class child(parent):
    def display(self):
        print("child method")
```

```
p= parent()
c= child()
c.show() # Works
c.display() # Works.

p.show() # Works.
p.display() # AttributeError.
```

6. If the parent has __init__() & the child does not, which __init__() run?

→ The parent's __init__() runs because the child inherits it when the child does not define it's own __init__().

e.g:-
```
class parent:
    def __init__(self):
        print("parent constructor")
class child(parent):
    pass
c=child()
```

7. If both parent & child have their own __init__() , which one runs? What problem occurs?

→ -When we create a child object, the child __init__() runs by default. The parent's __init__() does not automatically run.

e.g:-
```
class parent:
    def __init__(self):
        self.name="pratik"

    def show(self):
        print(self.name)
```

```
class child(parent):
    def __init__(self):
        self.age = 22
c = child()
c.show()
# This raised An AttributeError because self.name
was never created for this object.
```

⇒ Why? the child constructor did not call the parent's constructor, so the parent's name attribute was not initialized.

Soln:- Use super().__init__().

```
class child(parent):
    def __init__(self):
        super().__init__()
        self.age=22
c = child()
c.show().
# pratik.
```

8. Can a child access attribute created in the parent's __init__() using self?

→ Yes. A child object can access attribute initialized by the parent constructor, provided the parent constructor has run & the attributes are accessible.

⇒ ex:-

```
class parent:
    def __init__(self):
        self.name = "pratik"
class child(parent):
    def display(self):
        print(self.name)
c = child()
c.display() # pratik.
```

Note: If the child overrides --init--() Without calling super.--init--() the parent's attribute may not exist.

9. What if the parent & child have methods with same name?

→ When both classes define a method with the same name, the child object call uses the child's version by default.

ex:-

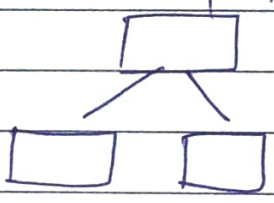
```
class parent:
    def show(self):
        print("parent show method")

class child(parent):
    def show(self):
        print("child show method")
c = child()
c.show()
# child show method.
```

So we call the parent's version explicity:
super().show()

10. What are the type of inheritance supported in python

→ 1. Single inheritance: A → B
2. Multiple inheritance = A + B → C
3) Multilevel inheritance = A → B → C
4) Hierachical inheritance.



A → B & A → C

Date .....
Page .....

5> Hybrid inheritance.
A combine of two or more inheritance type.